

WORKPLACE NINJAS AUSTRALIA · MELBOURNE 2026

From clicks to code to AI agents

Graph X-Ray, Lokka and graph.pm: why they exist, how they work under the hood, and how to use them day to day.

Merill Fernando · merill.net

Graph X-Ray

- Graph X-Ray for Edge
- Graph X-Ray for Chrome
- Graph X-Ray for Firefox
- Graph X-Ray on GitHub

Lokka

- lokka.dev
- Install Lokka
- Lokka on GitHub

graph.pm

- graph.pm
- graph.pm skill on GitHub

Community

- nodoc by Nathan McNulty
- nodoc on GitHub

Microsoft Graph

- Graph Explorer
- Use the Microsoft Graph API
- JSON batching in Microsoft Graph
- Invoke-MgGraphRequest docs
- Find-MgGraphCommand docs
- msgraph-metadata (OpenAPI)
- microsoft-graph-docs-contrib

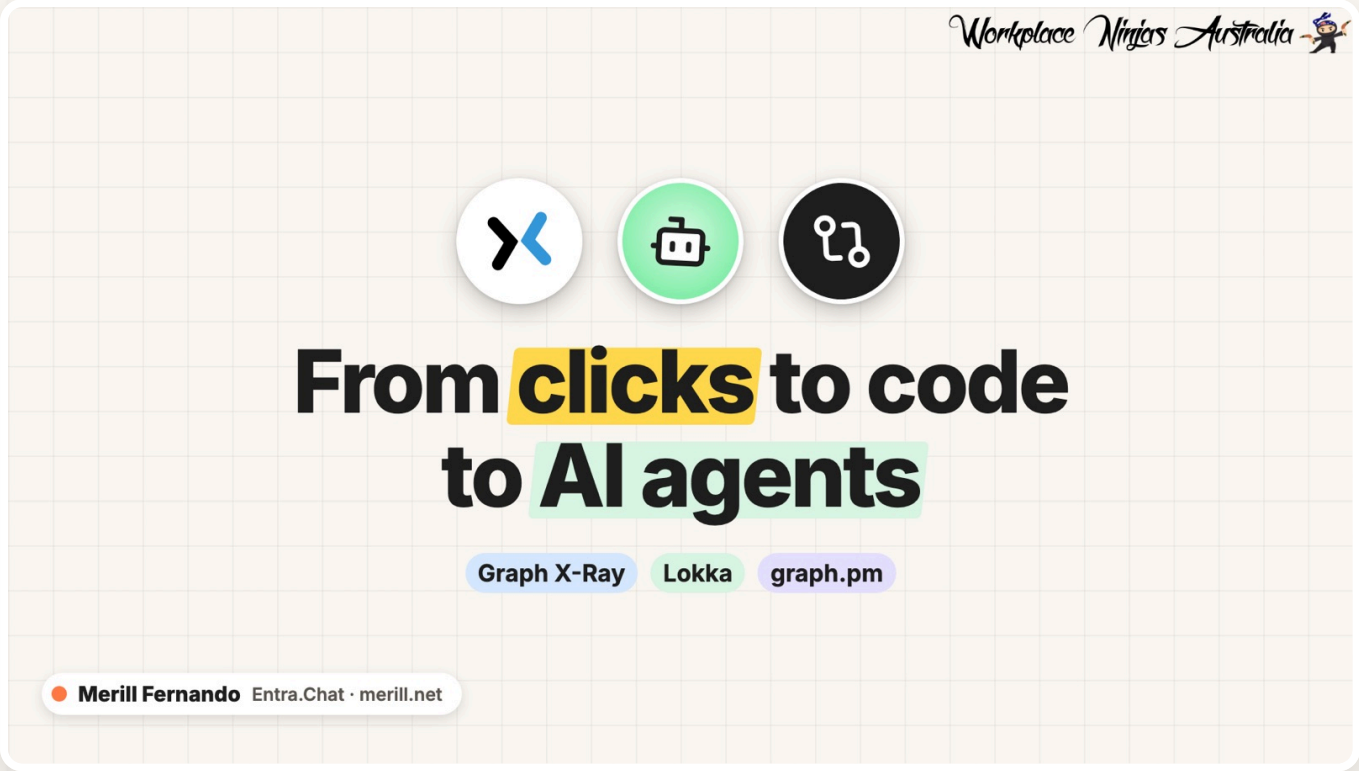
Standards

- Model Context Protocol
- MCP Apps
- Agent Skills

Merill

- merill.net
- Entra.Chat

From clicks to code to AI agents



Three tools, one goal: make Microsoft Graph approachable, first for people and now for AI agents.

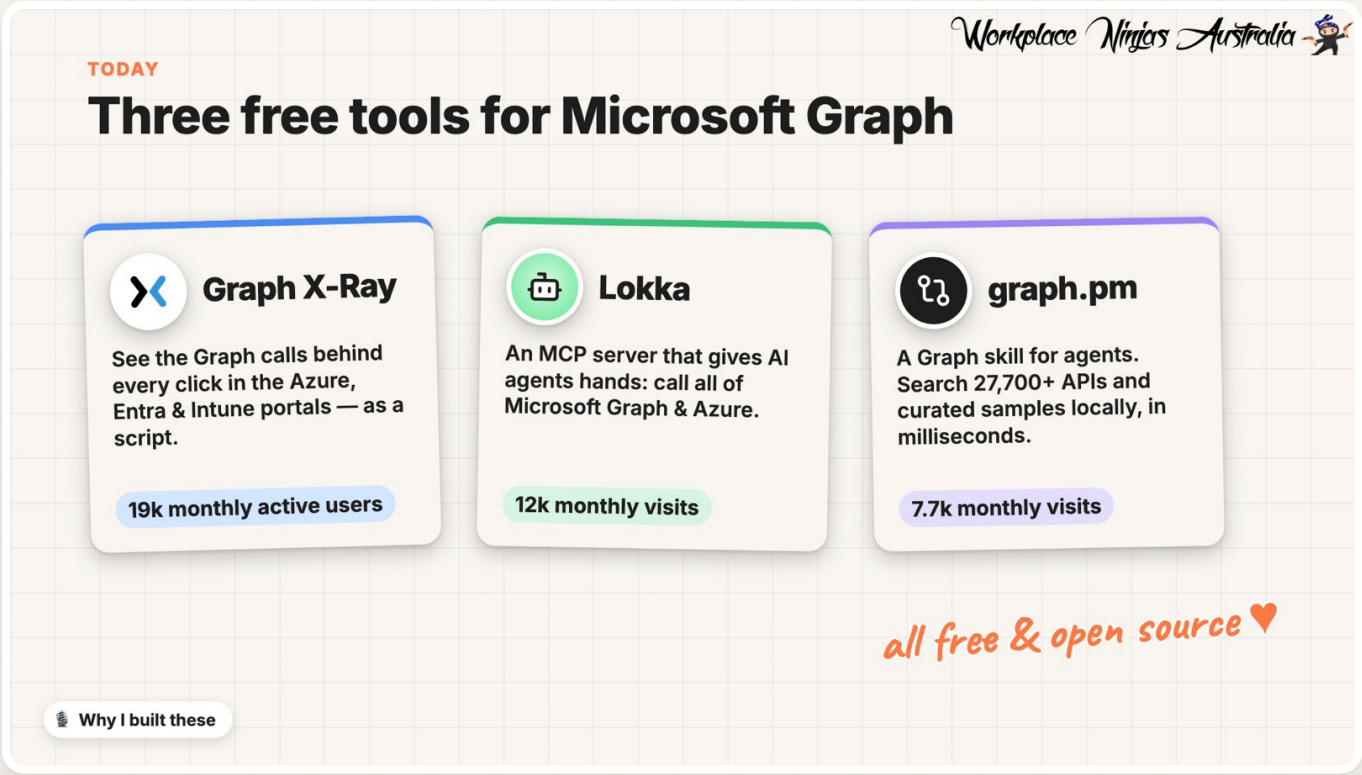
Graph X-Ray shows you the Graph calls behind the admin portals. **Lokka** lets an AI agent make those calls for you. **graph.pm** makes sure the agent picks the right call.

LINKS

merill.net
merill.net

Entra.Chat
entra.chat

Three free tools for Microsoft Graph



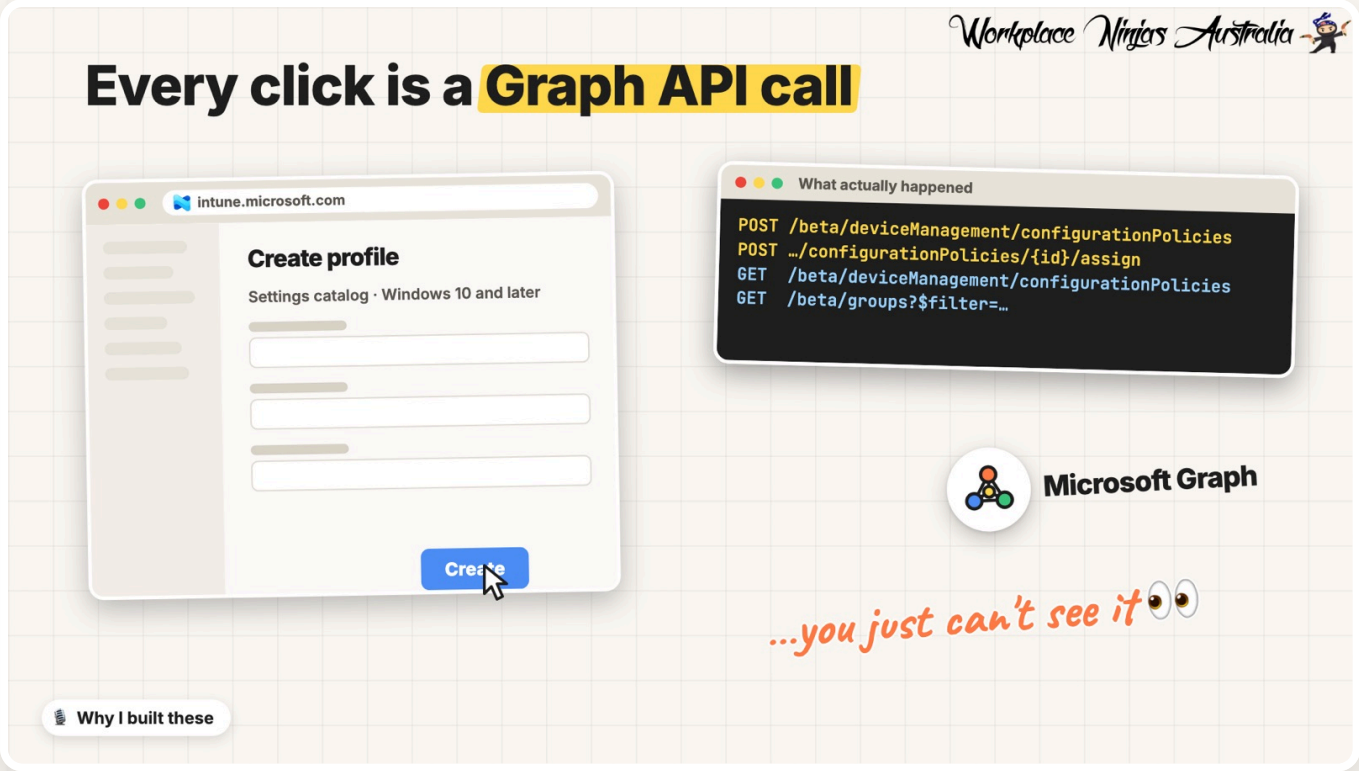
All three are free and open source. Graph X-Ray is a browser extension used by about 19,000 people a month. Lokka is an MCP server; lokka.dev gets about 12,000 visits a month. graph.pm is a Graph skill for AI agents; graph.pm gets about 7,700 visits a month.

Graph X-Ray is AGPL-3.0. Lokka and the graph.pm skill are MIT-licensed.

LINKS

- Graph X-Ray on GitHub — github.com/merill/graphxray
- lokka.dev — lokka.dev
- graph.pm — graph.pm

Every click is a Graph API call



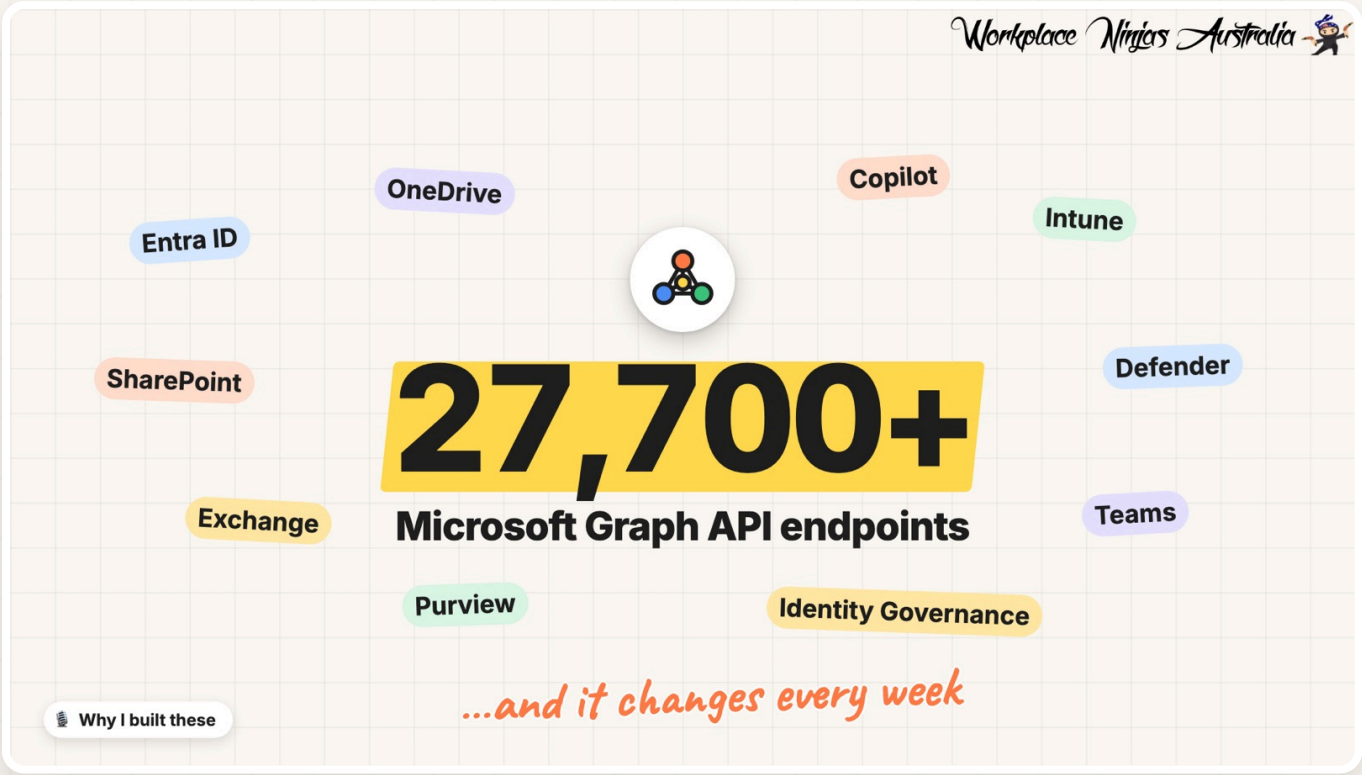
The Azure, Entra and Intune admin portals are web apps that call Microsoft Graph. One click, like creating an Intune settings catalog profile, can send several requests: create the policy, assign it, refresh lists, look up groups.

Anything the portal does, you can automate, as long as you can see those calls.

LINKS

Settings catalog policy resource
`learn.microsoft.com/en-us/graph/api/resources/intune-deviceconfigv2-devicemanagementconfigurationpolicy`

Microsoft Graph is huge



The Microsoft Graph beta OpenAPI description lists more than 27,700 endpoints across Entra, Intune, Exchange, Teams, SharePoint, Defender, Purview and more, and it changes every week.

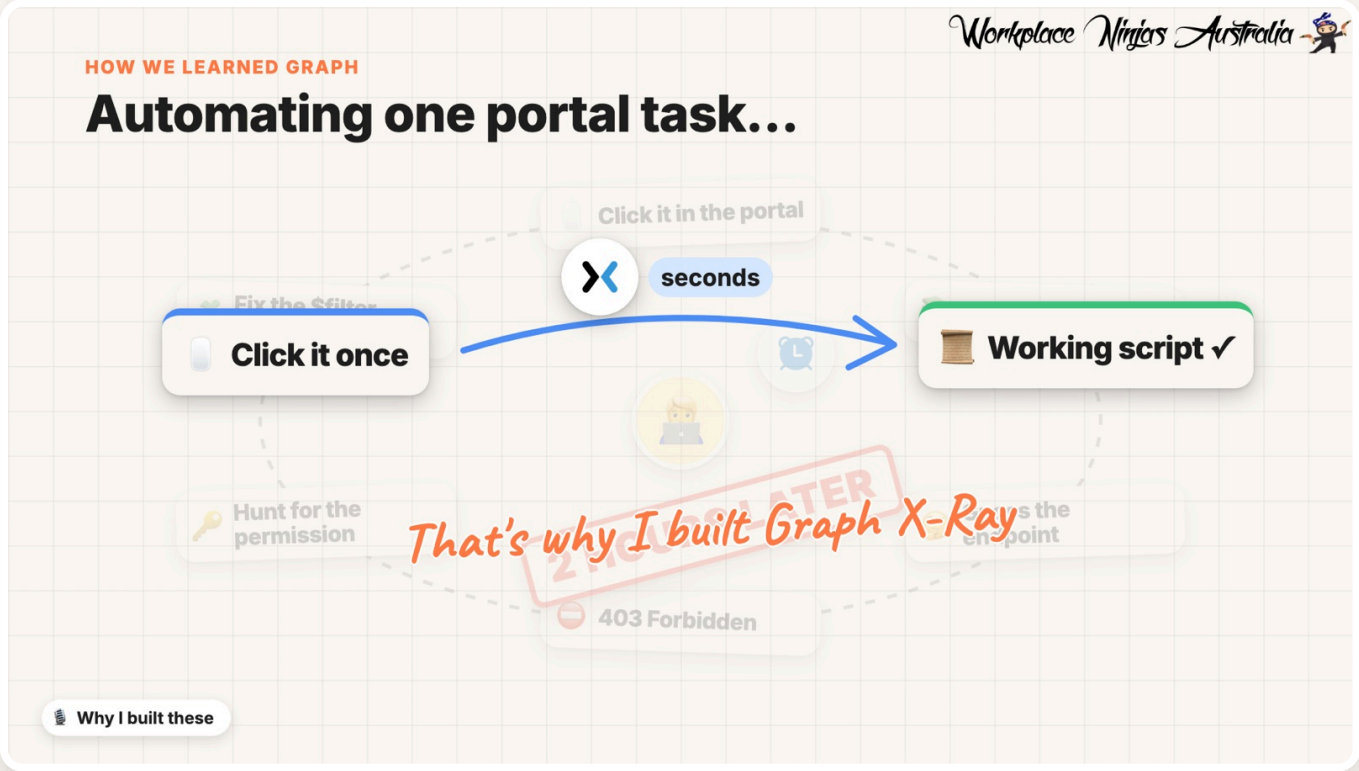
That's why docs, blog posts and AI models trained months ago are often out of date.

LINKS

Use the Microsoft Graph API
learn.microsoft.com/en-us/graph/use-the-api

msgraph-metadata (OpenAPI)
github.com/microsoftgraph/msgraph-metadata

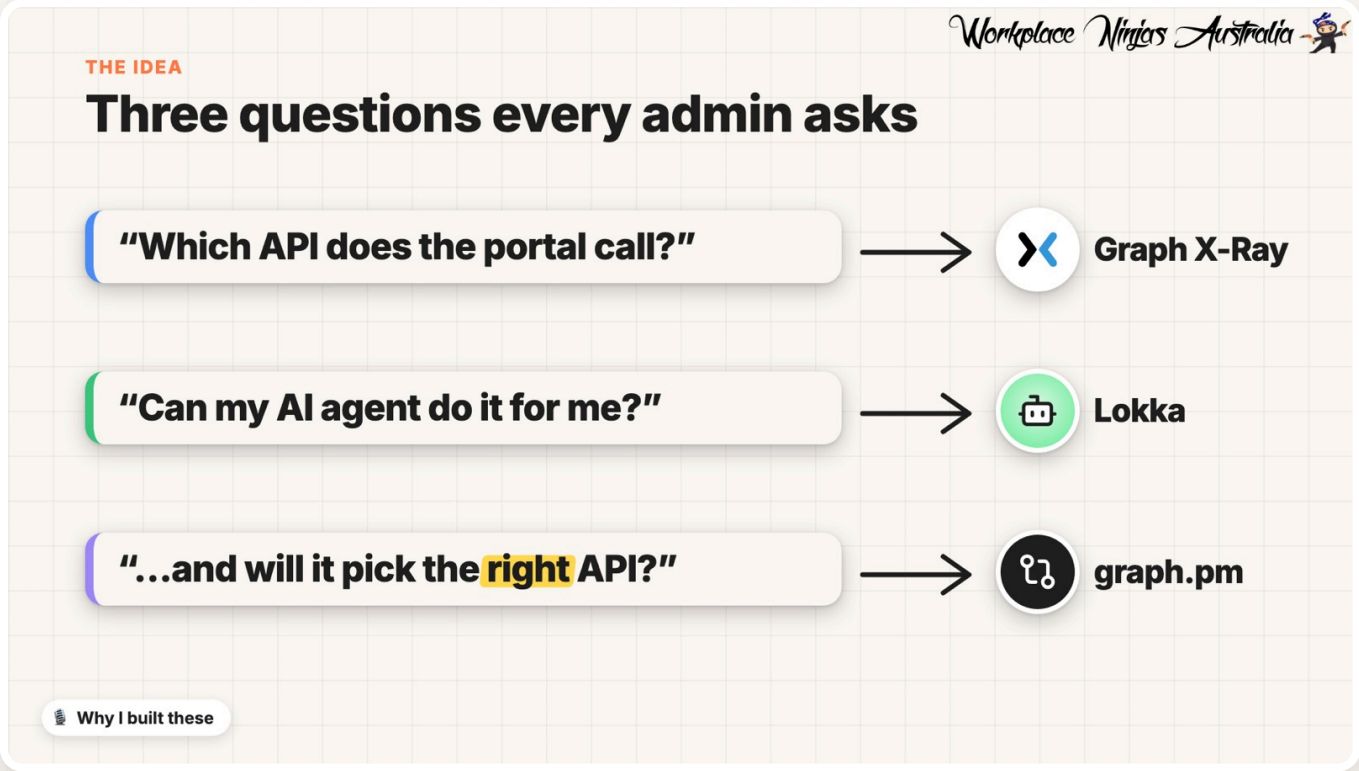
The old way to automate a portal task



The usual way to automate a portal task: do it in the portal, search the docs, guess the endpoint, hit a 403, hunt for the permission, fix the `$filter` syntax. Repeat.

Graph X-Ray removes the loop. Do the task once in the portal and copy the script it generates.

Three questions every admin asks

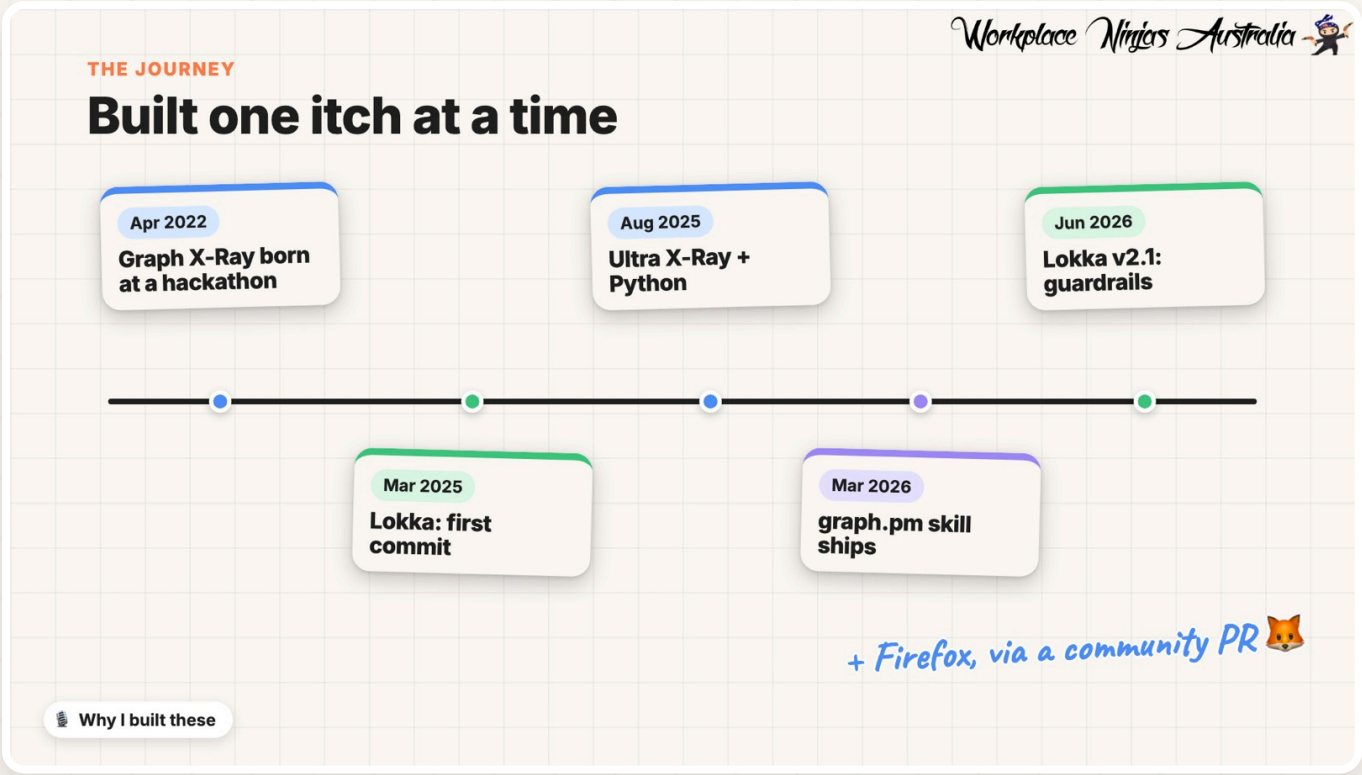


Which API does the portal call? Graph X-Ray.

Can my AI agent do it for me? Lokka.

Will it pick the right API, permission and syntax? graph.pm.

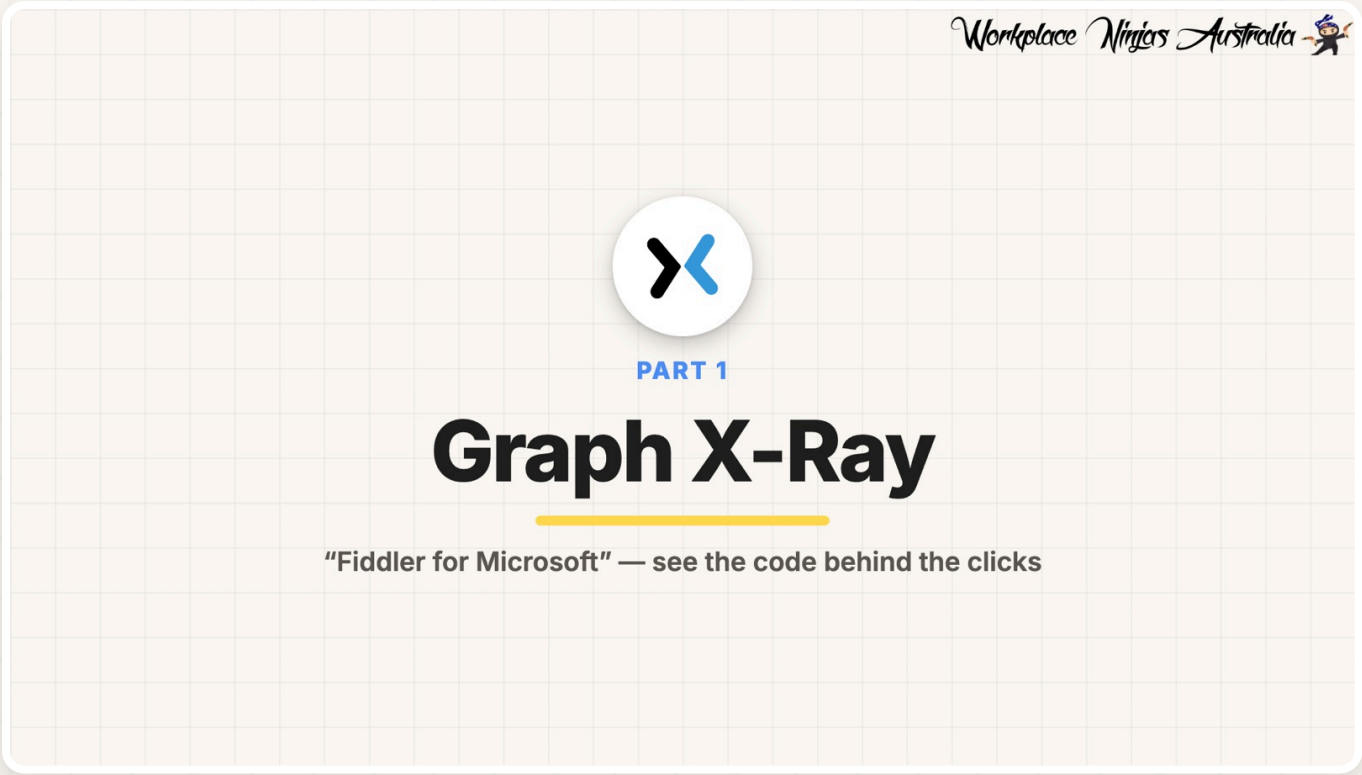
Built one itch at a time



Graph X-Ray started at a 2022 hackathon (with Eunice, Dhruv, Clement and Monica). Lokka's first commit was in March 2025. Graph X-Ray added Ultra X-Ray mode and Python in August 2025. graph.pm shipped in March 2026, and Lokka v2.1 added guardrails in June 2026.

Firefox support for Graph X-Ray came from a community pull request by @jorgeasaurus.

Part 1 — Graph X-Ray

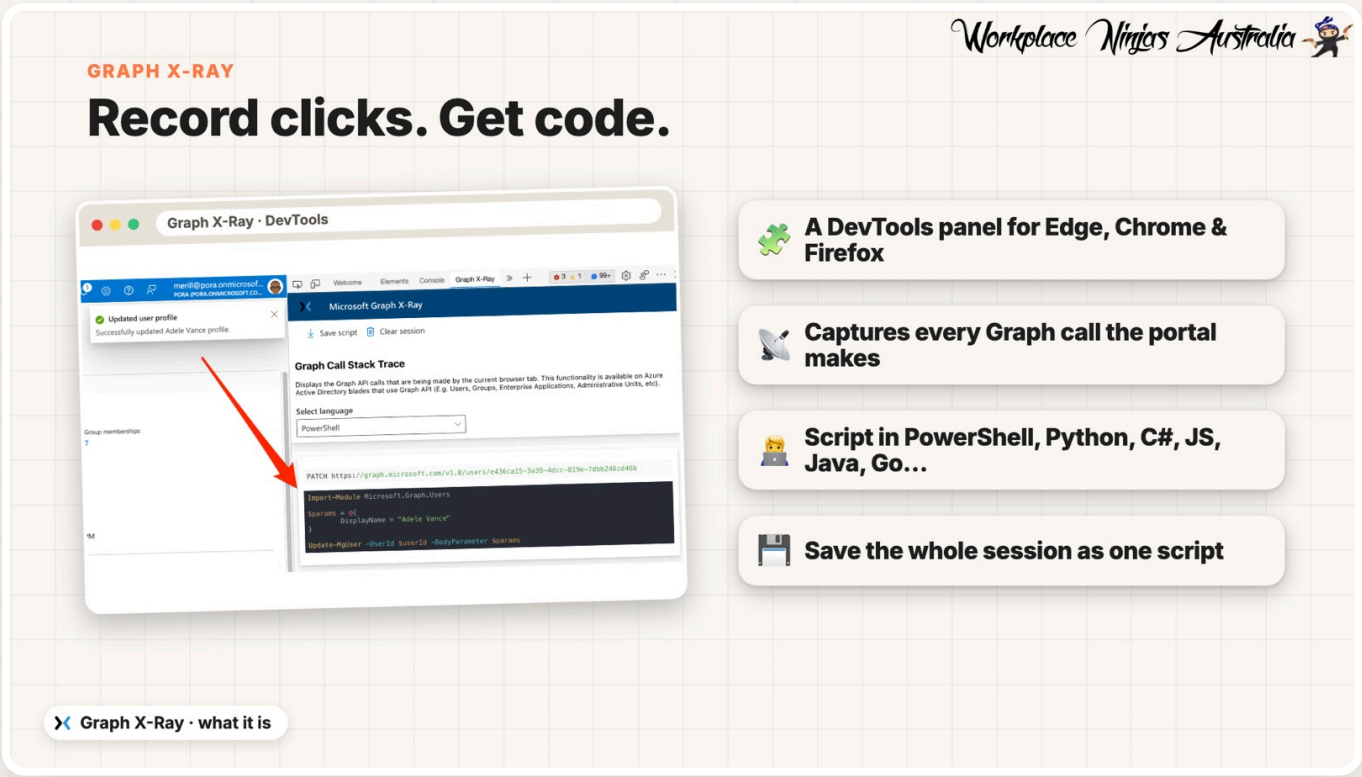


Part 1: Graph X-Ray, sometimes called “Fiddler for Microsoft”. It shows you the code behind the clicks.

LINKS

Graph X-Ray on GitHub
`github.com/merill/graphxray`

Record clicks. Get code.



Graph X-Ray adds a **Graph X-Ray** tab to your browser's developer tools (F12). As you work in a Microsoft admin portal, every Graph call appears there as a ready-to-run snippet.

Snippets are available in PowerShell (Graph SDK cmdlets or `Invoke-MgGraphRequest`), Python, C#, JavaScript, Java, Go and Objective-C, plus the raw REST URL. **Save script** downloads the whole session as one file.

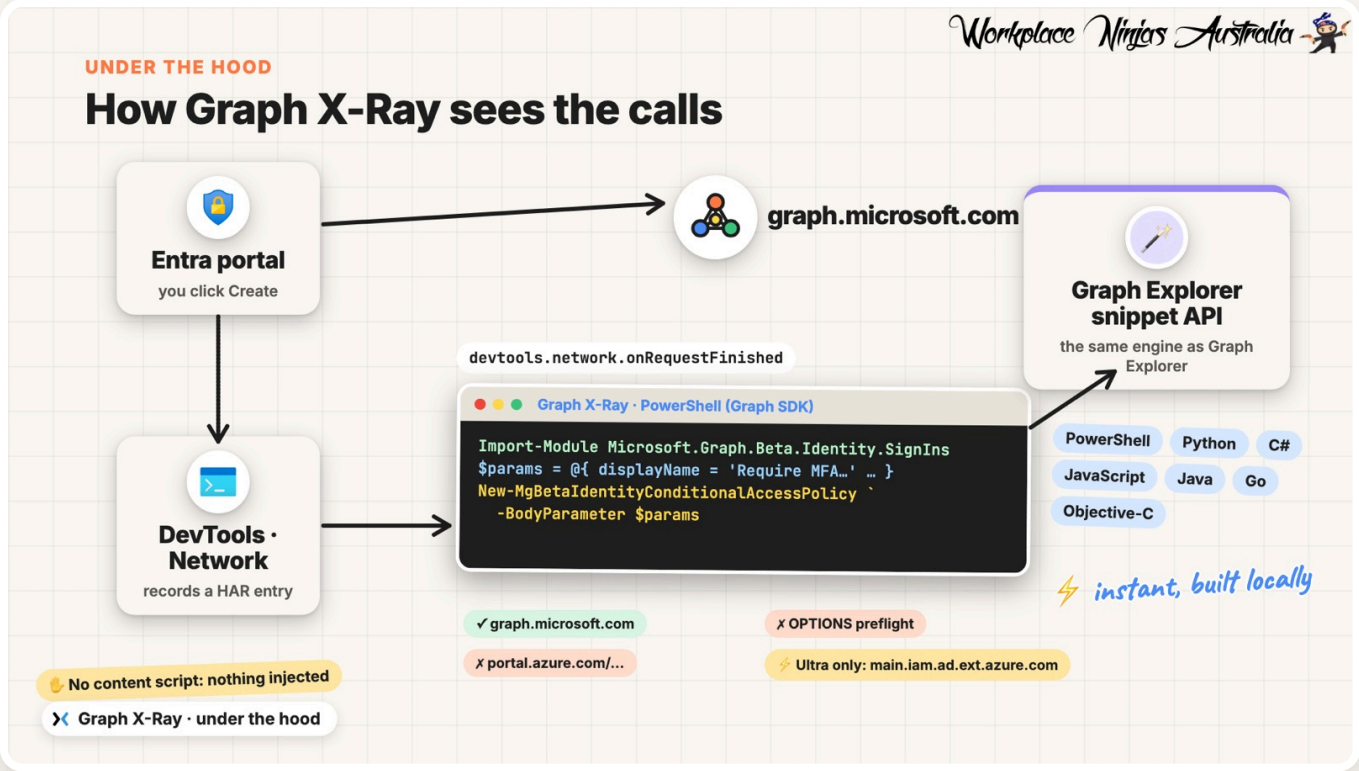
LINKS

Graph X-Ray for Edge
`microsoftedge.microsoft.com/addons/detail/graph-xray/oplgganppgjhpihgciiifejplnnpodak`

Graph X-Ray for Chrome
`chrome.google.com/webstore/detail/graph-x-ray/gdhbldfajbedclijgcmmmobdbnjhnpdh`

Graph X-Ray for Firefox
`addons.mozilla.org/en-US/firefox/addon/graph-xray`

How Graph X-Ray sees the calls



1. You click in the portal, and the portal's own JavaScript calls graph.microsoft.com.

2. The DevTools Network panel records the request as a HAR entry. Graph X-Ray reads that same feed, so nothing is injected into the page. That's also why DevTools must be open.

3. A filter keeps Microsoft Graph hosts (every cloud, including GCC High, DoD and China) and drops OPTIONS preflights and other portal traffic.

4. A PowerShell snippet using `Invoke-MgGraphRequest` appears instantly, generated in the browser.

5. The request is also sent to the Graph Explorer snippet service, which returns Graph SDK code (for example `New-MgBetaIdentityConditionalAccessPolicy`) and the other languages.

LINKS

Graph Explorer
developer.microsoft.com/en-us/graph/graph-explorer

Two calls do the heavy lifting



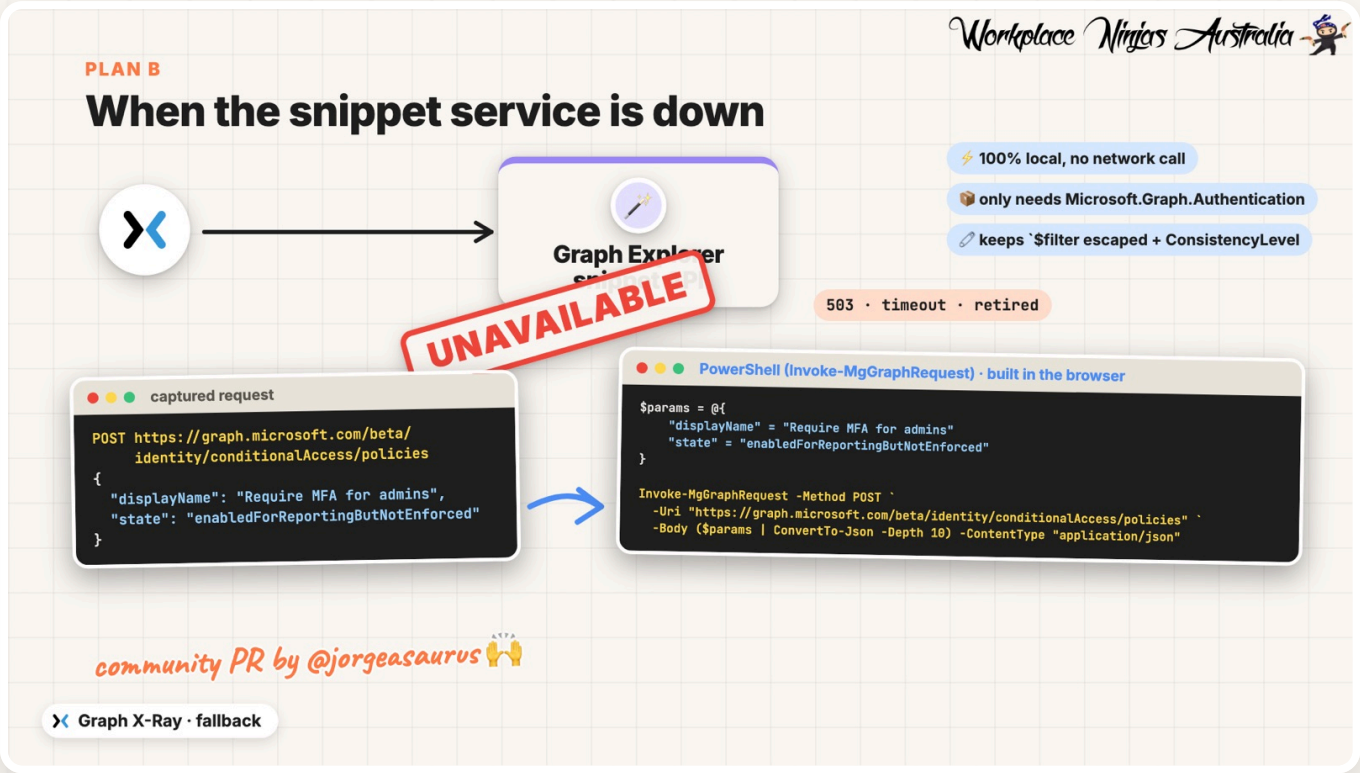
Capture is a single listener: `chrome.devtools.network.onRequestFinished`, filtered by host.

To generate code, the request is turned back into raw HTTP text and POSTed to the snippet service with `content-type: application/http` and a `?lang=` parameter. Your access token is never sent; only the method, path, host and request body are.

LINKS

Graph X-Ray on GitHub
github.com/merill/graphxray

When the snippet service is down



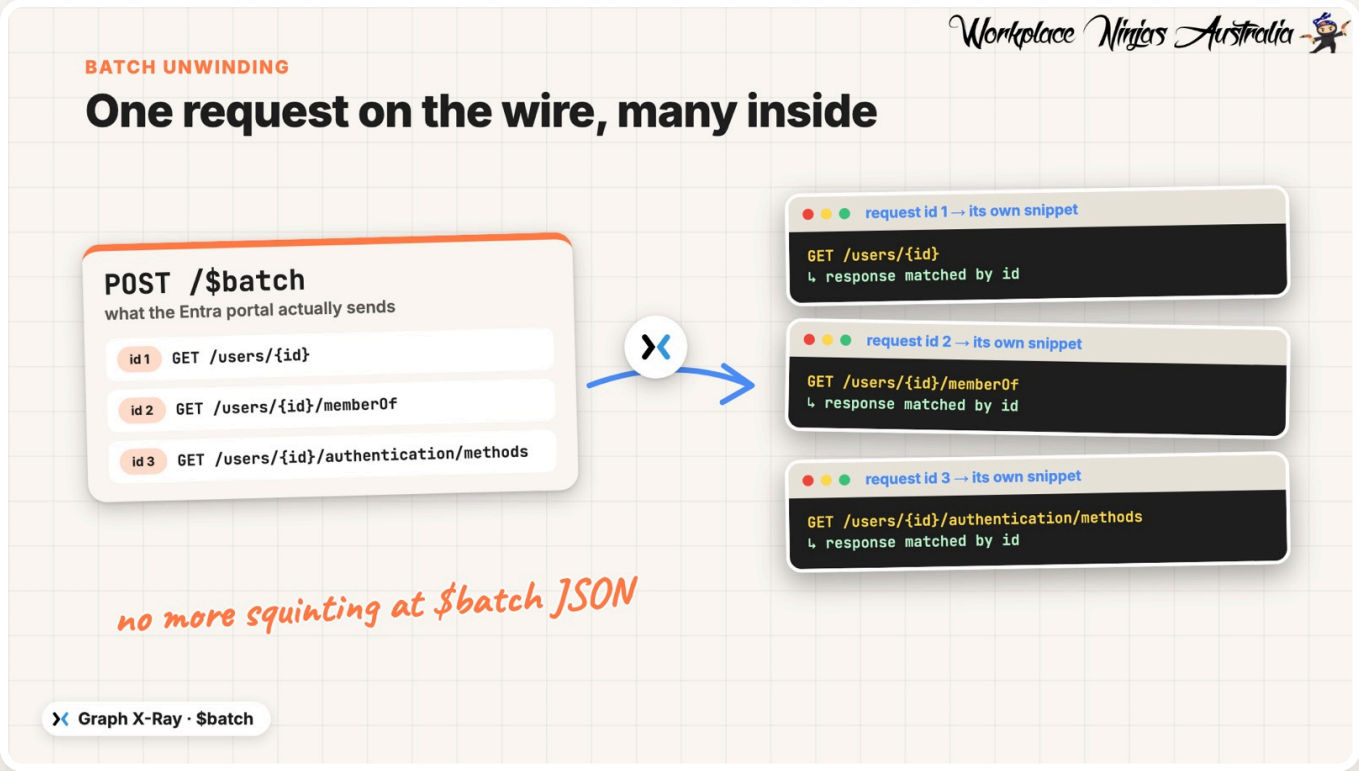
If the snippet service fails (an error, a timeout, or one day being retired), Graph X-Ray still gives you PowerShell. It turns the captured JSON body into a `$params` hashtable and writes an `Invoke-MgGraphRequest` call with the full URL, method and `-Body ($params | ConvertTo-Json -Depth 10)`.

This runs entirely in your browser and only needs the `Microsoft.Graph.Authentication` module. It keeps `$` in URLs escaped (`` `$filter ``) and keeps the `ConsistencyLevel: eventual` header for advanced queries. To always use it, pick **PowerShell (Invoke-MgGraphRequest)** in the language list. Thanks to @jorgeasaurus for this contribution.

LINKS

Invoke-MgGraphRequest docs
learn.microsoft.com/en-us/powershell/module/microsoft.graph.authentication/invoke-mggraphrequest

Batch unwinding

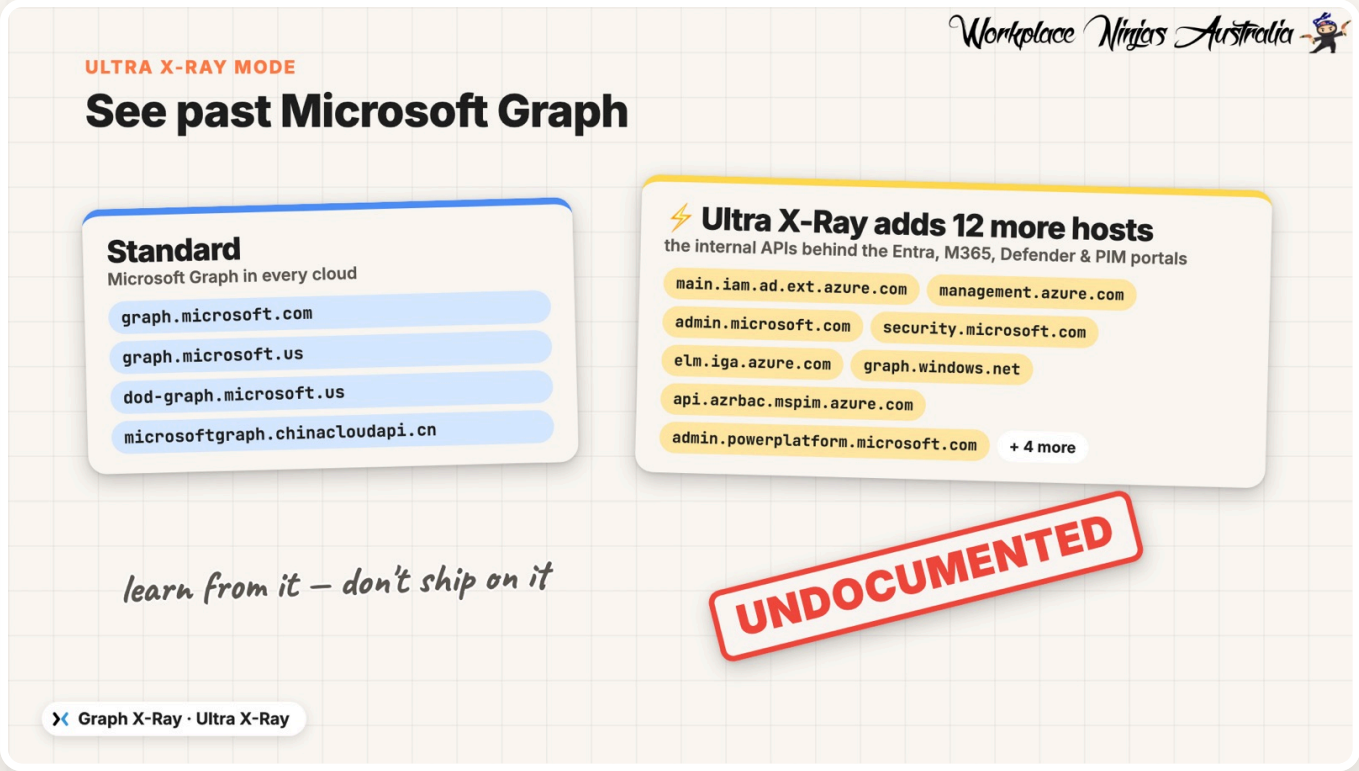


Portals often bundle many requests into one `POST /$batch`. Graph X-Ray unpacks the batch, creates a snippet for each inner request, and matches each one to its response by `id`.

LINKS

JSON batching in Microsoft Graph
learn.microsoft.com/en-us/graph/json-batching

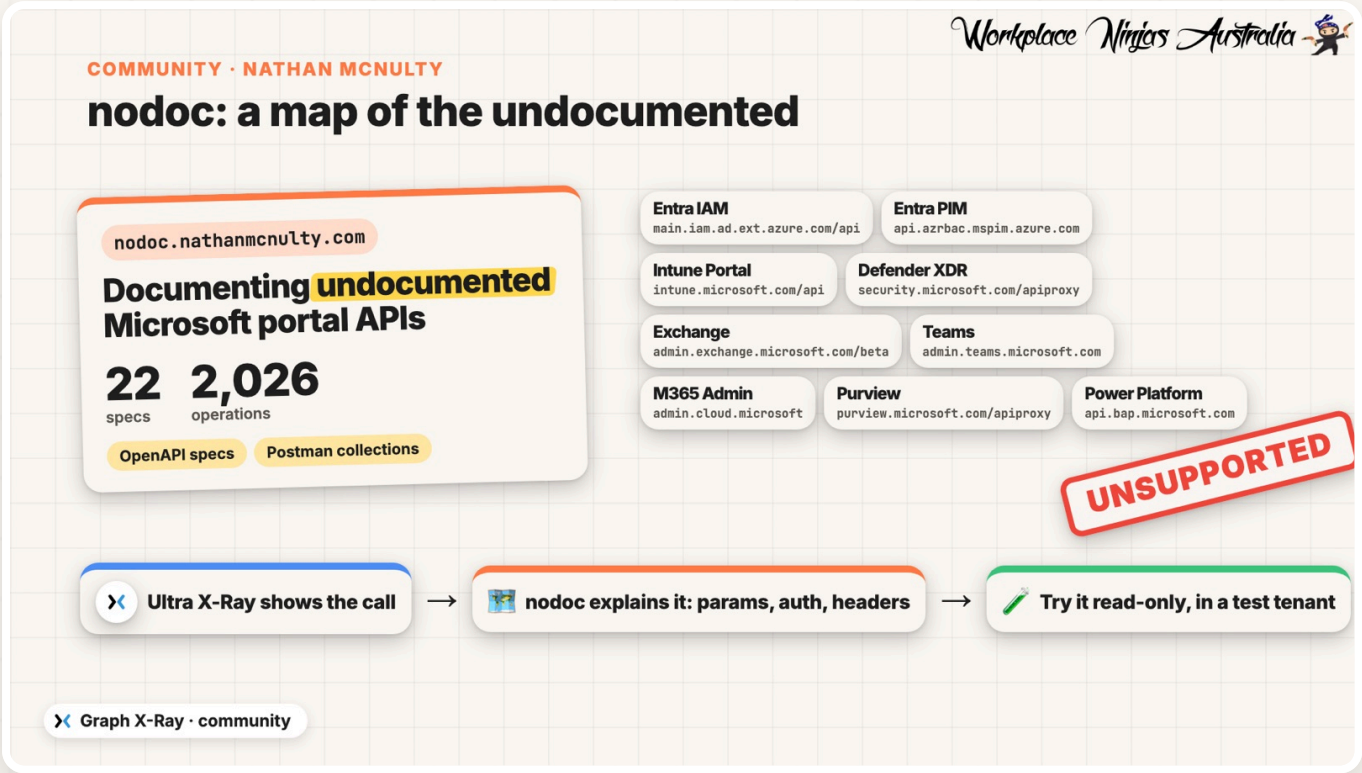
Ultra X-Ray mode



Standard mode shows Microsoft Graph calls. **Ultra X-Ray** also shows calls to 12 internal portal APIs, such as `main.iam.ad.ext.azure.com` (Entra), PIM, Identity Governance, the Microsoft 365 admin center, Defender, Power Platform and Azure Resource Manager.

These APIs are undocumented and unsupported by Microsoft, so Graph X-Ray shows the request and response but doesn't generate code. Use them to learn, not in production.

nodoc: a map of the undocumented



Nathan McNulty's **nodoc** project documents undocumented Microsoft portal APIs: 22 specs and 2,026 operations at the time of writing, published as OpenAPI specs and Postman collections.

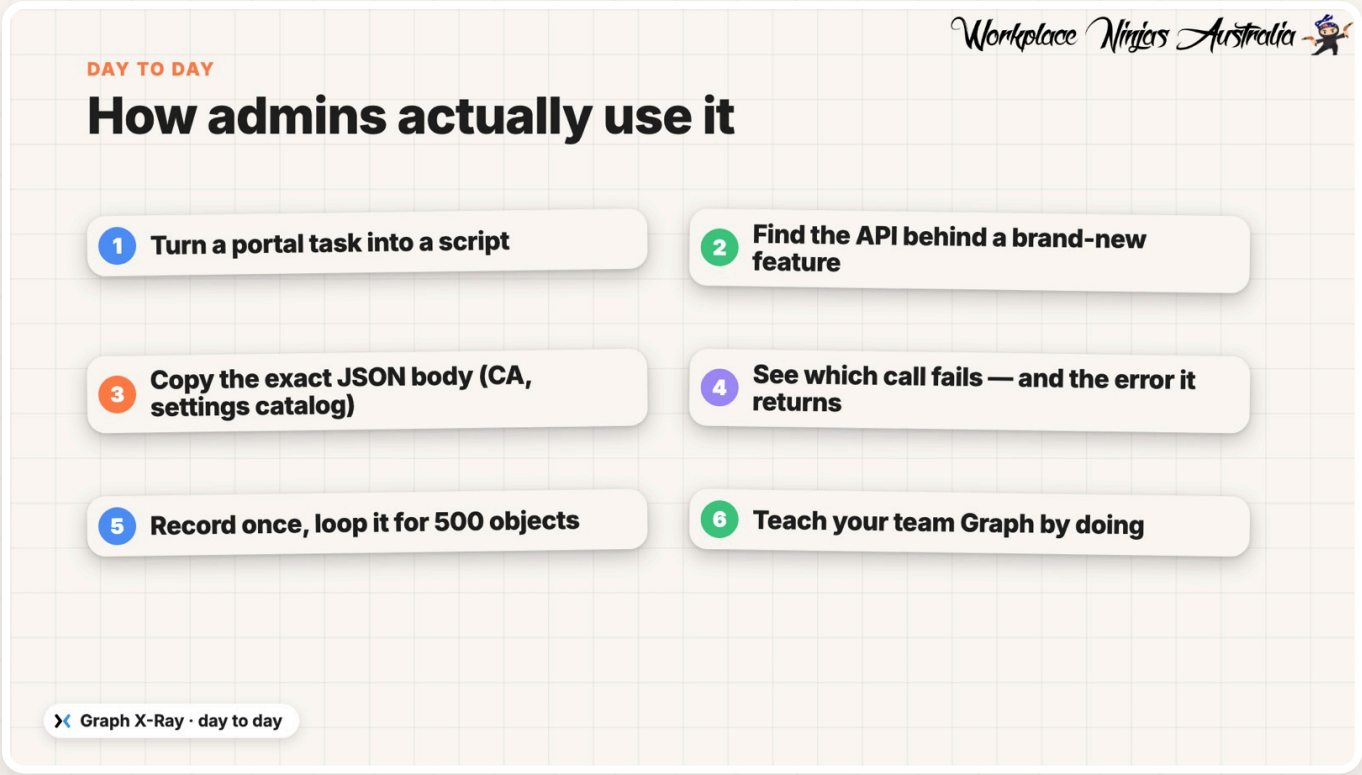
It covers the same hosts Ultra X-Ray shows, including Entra IAM, PIM, Intune, Defender XDR, Exchange, Teams, the Microsoft 365 admin center, Purview and Power Platform. A good workflow: see the call in Ultra X-Ray, look it up in nodoc to understand its parameters and how it authenticates, then try it read-only in a test tenant. These APIs can change without notice.

LINKS

nodoc by Nathan McNulty
nodoc.nathanmcnulty.com

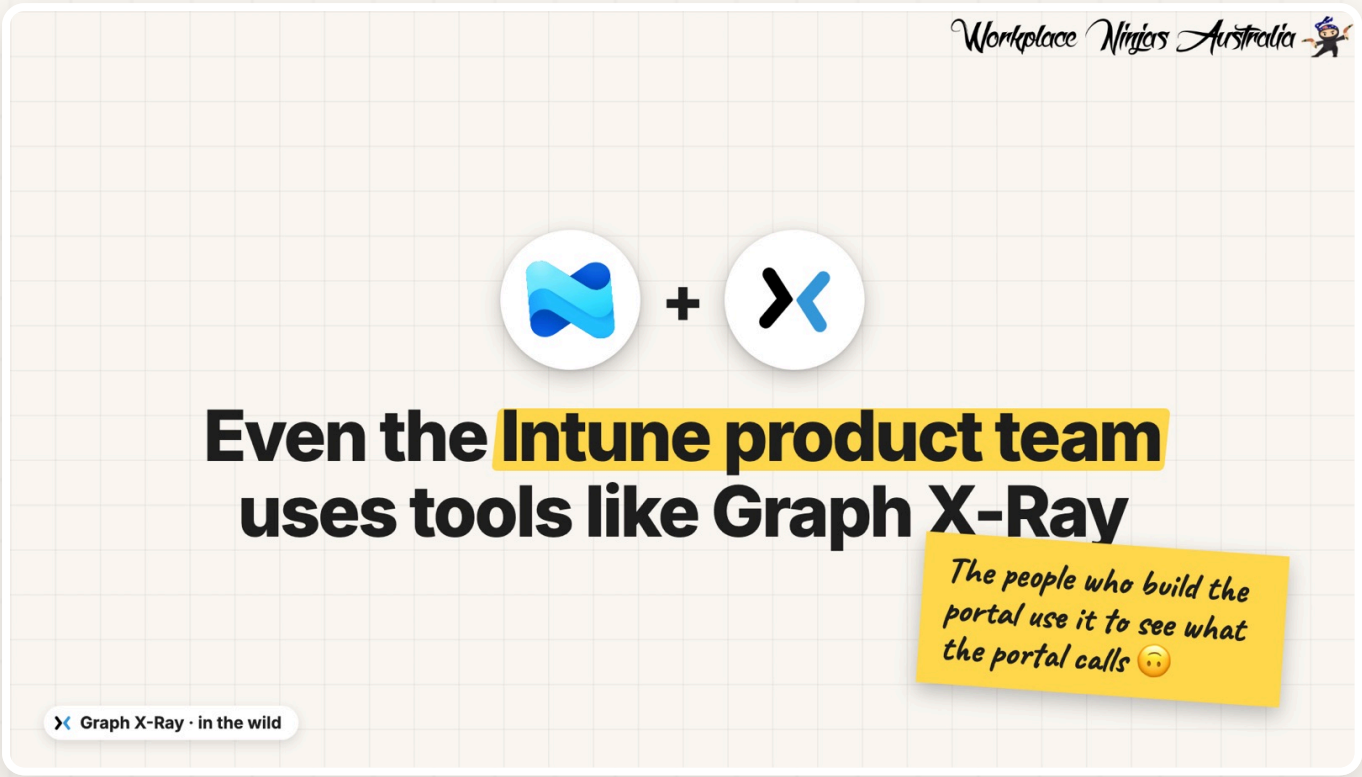
nodoc on GitHub
github.com/nathanmcnulty/nodoc

How admins actually use it



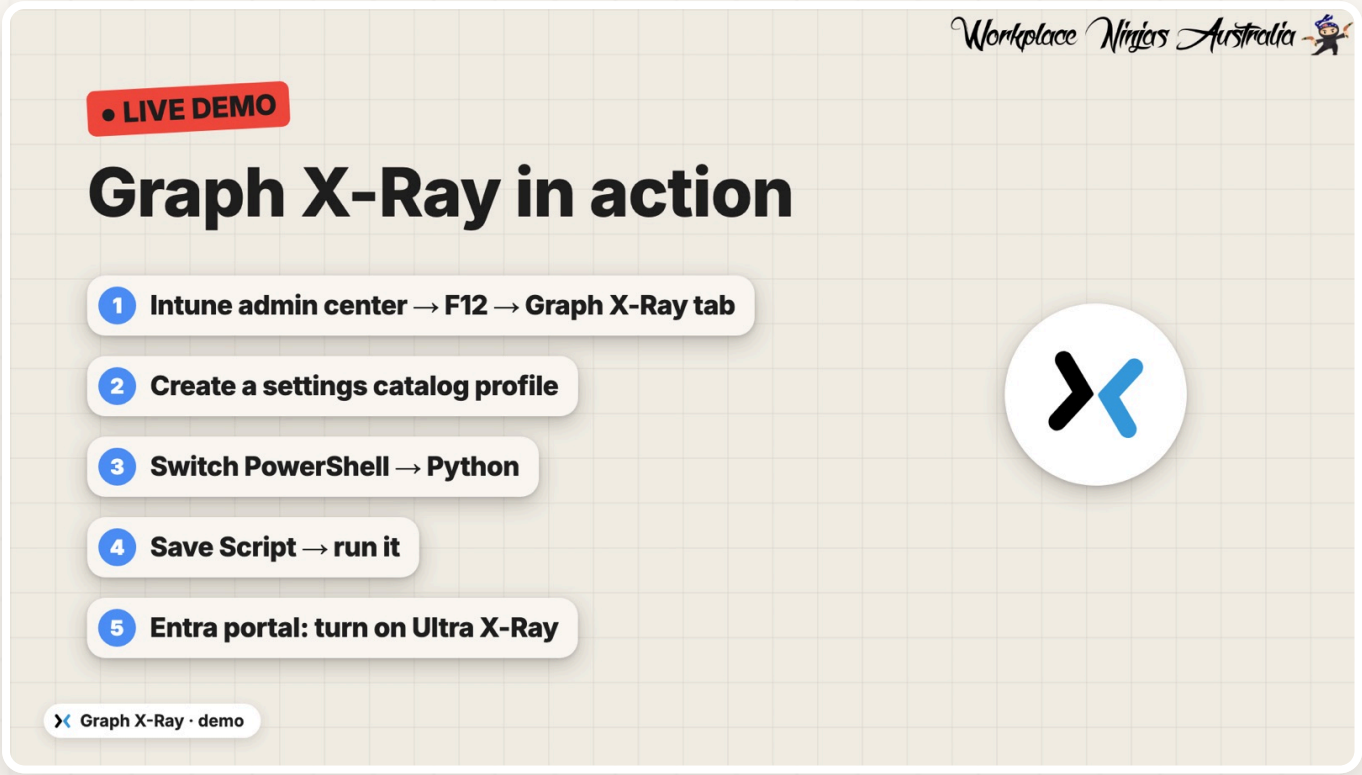
Common uses: turn a portal task into a script; find the API behind a brand-new feature before it's documented; copy the exact JSON body for complex objects like Conditional Access policies or settings catalog profiles; see which call failed and the error it returned; record one change and loop it over hundreds of objects; and teach Graph by doing.

Even the Intune product team uses it



Even the Intune product team uses tools like Graph X-Ray to see what the portal calls.

Demo: Graph X-Ray



Try it yourself: install the extension, open the Intune or Entra admin center, press F12 and choose the Graph X-Ray tab. Make a change, switch languages, then use **Save script**. In the Entra admin center, turn on Ultra X-Ray to see the internal calls.

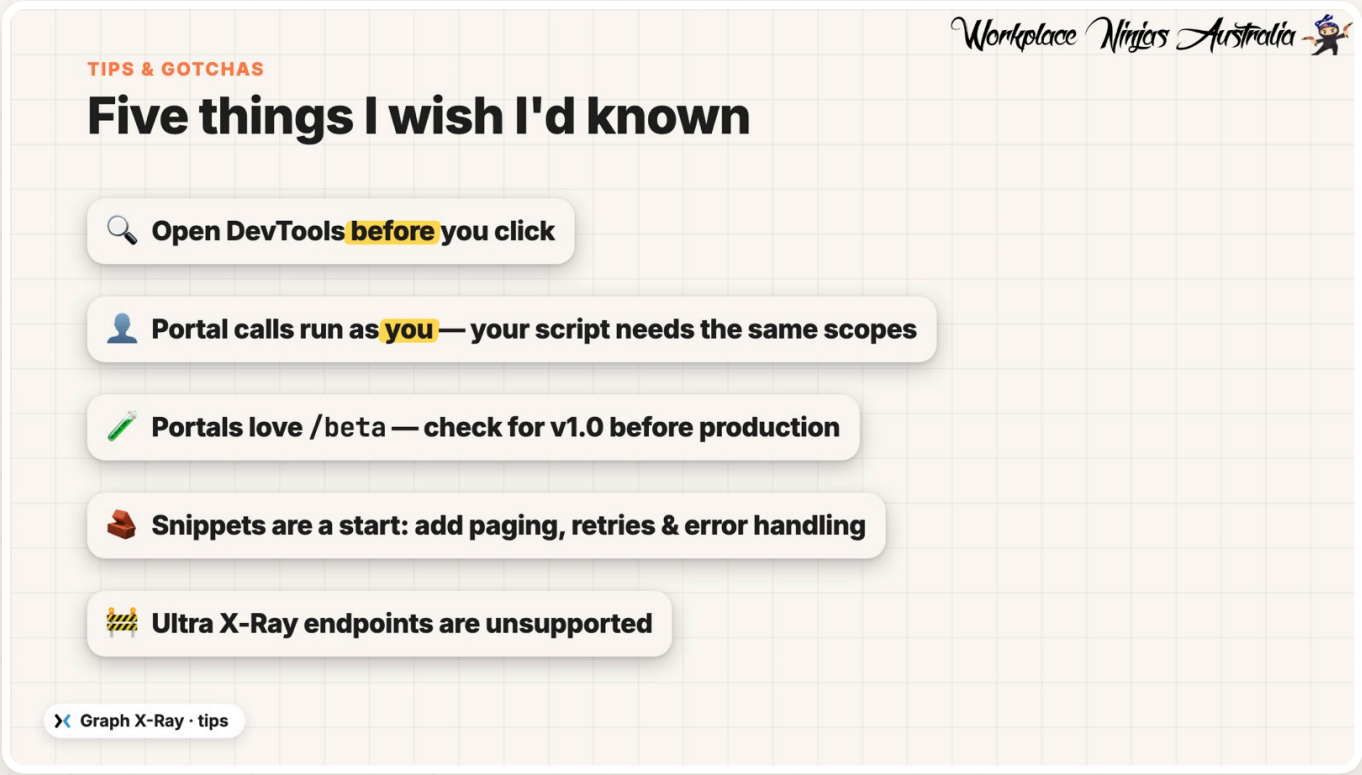
LINKS

Graph X-Ray for Edge
`microsoftedge.microsoft.com/addons/detail/graph-xray/oplgganppgjhpihgciiifejplnnpodak`

Graph X-Ray for Chrome
`chrome.google.com/webstore/detail/graph-x-ray/gdhbldfajbedclijgcmmmobdbnjhnpdh`

Graph X-Ray for Firefox
`addons.mozilla.org/en-US/firefox/addon/graph-xray`

Tips & gotchas

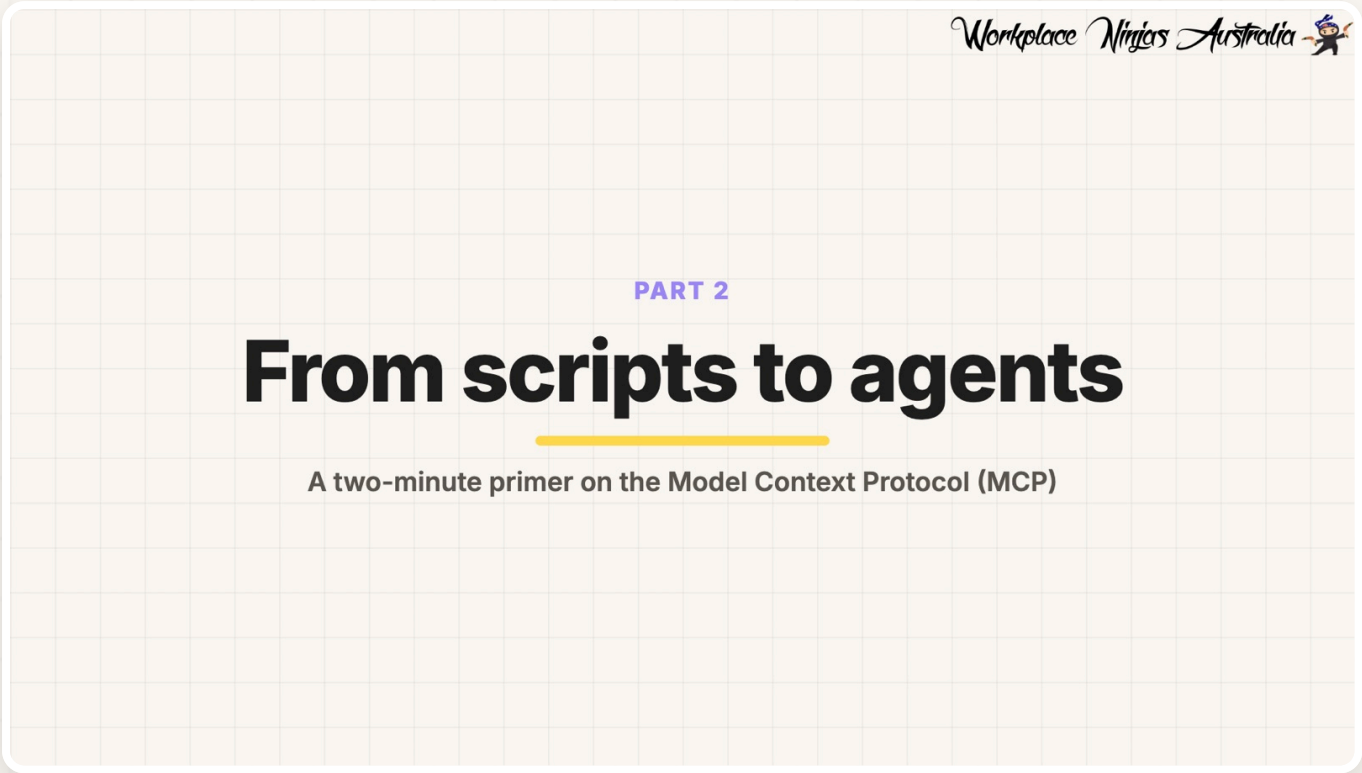


Open DevTools **before** you click. Portal calls use **your** delegated permissions, so connect with the scopes each call needs; `Find-MgGraphCommand` or graph.pm will tell you which. Portals use `/beta` heavily, so check for a v1.0 equivalent before production. A snippet is one call: add paging, retries for throttling (429) and error handling. Ultra X-Ray endpoints can change at any time.

LINKS

Find-MgGraphCommand docs
`learn.microsoft.com/en-us/powershell/module/microsoft.graph.authentication/find-mggraphcommand`

Part 2 — From scripts to agents

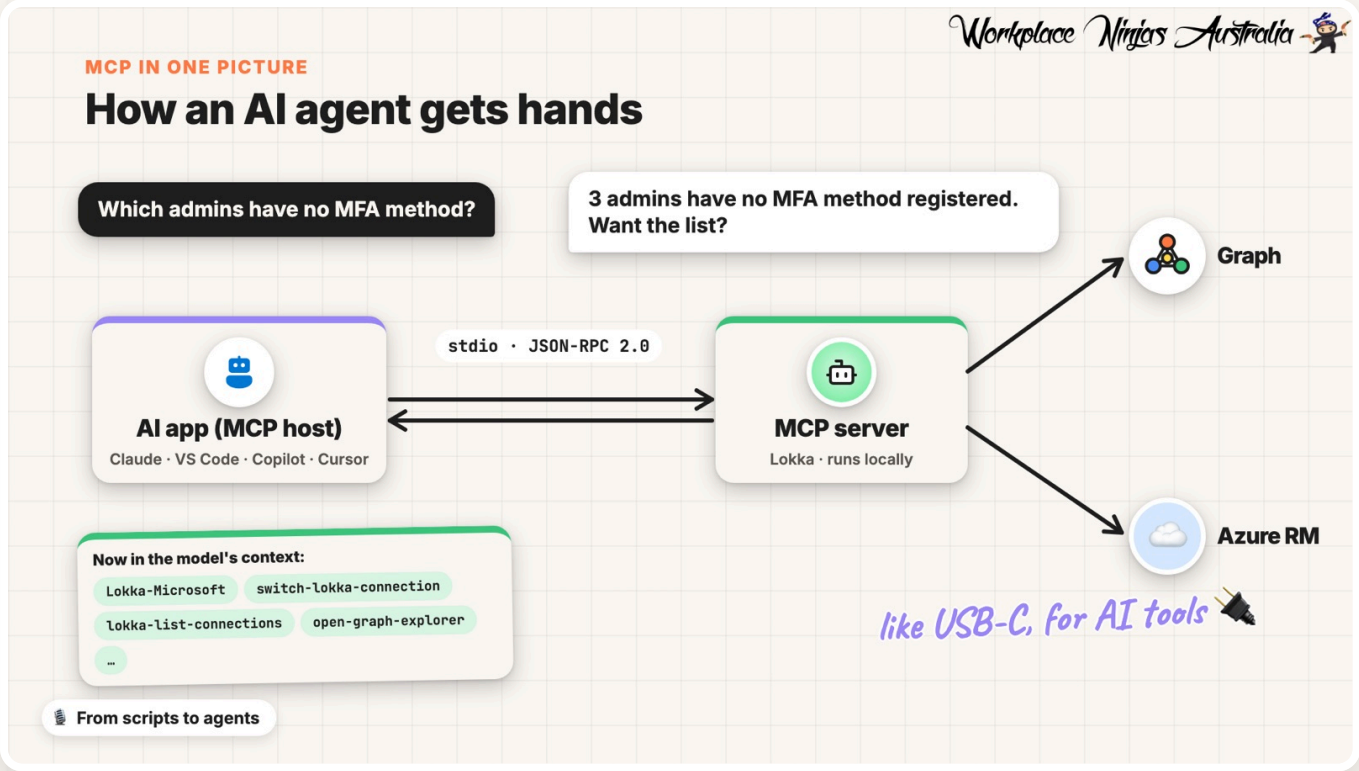


Part 2: from scripts to agents. To let an AI agent call Microsoft Graph for you, it needs a tool, and MCP is the standard way to give it one.

LINKS

Model Context Protocol
modelcontextprotocol.io

How an AI agent gets hands (MCP)



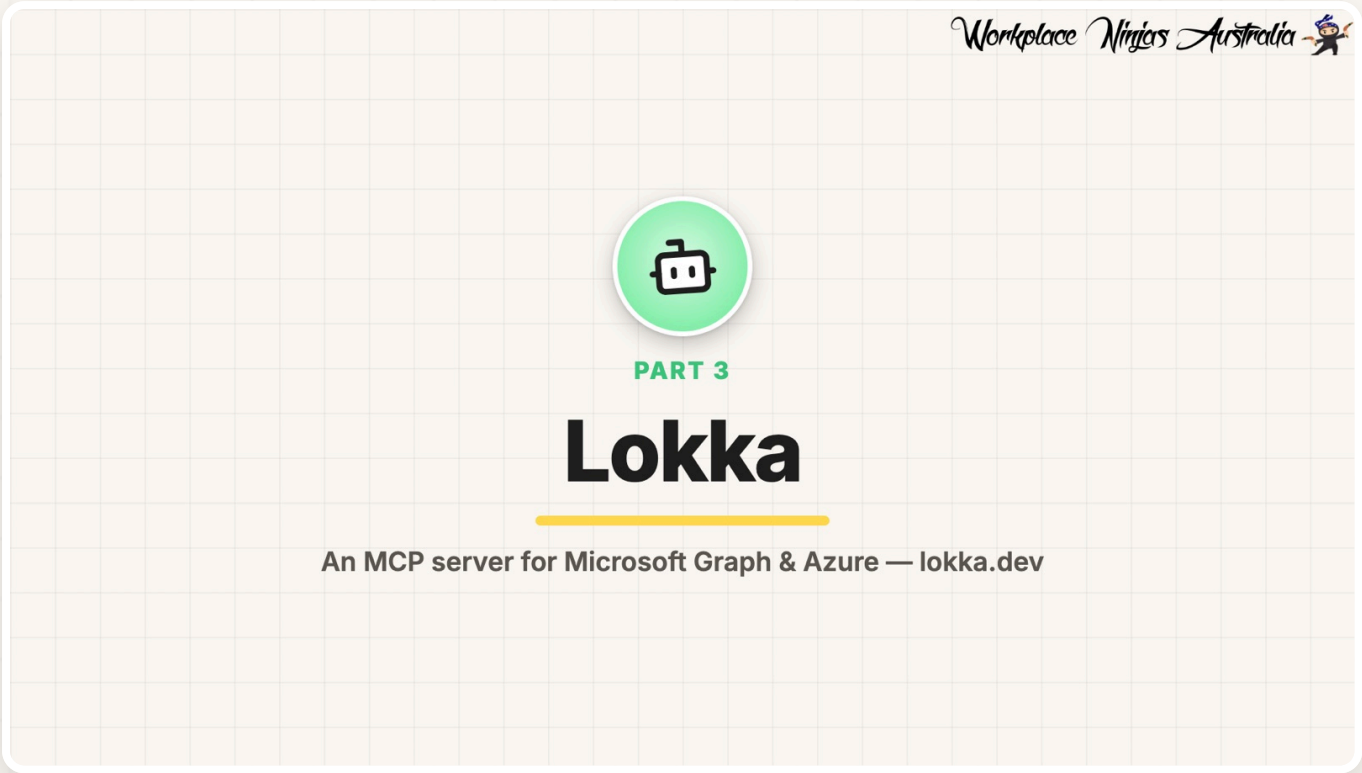
The **Model Context Protocol (MCP)** connects AI apps (the host: Claude, VS Code, GitHub Copilot, Cursor and others) to tools (MCP servers). Lokka runs locally and talks to the host over stdio using JSON-RPC 2.0.

When it connects, the host asks the server for its tools (`tools/list`) and their input schemas go into the model's context. When you ask a question, the model chooses a tool and sends `tools/call` ; Lokka calls Graph and returns the JSON for the model to summarise.

LINKS

Model Context Protocol
`modelcontextprotocol.io`

Part 3 — Lokka



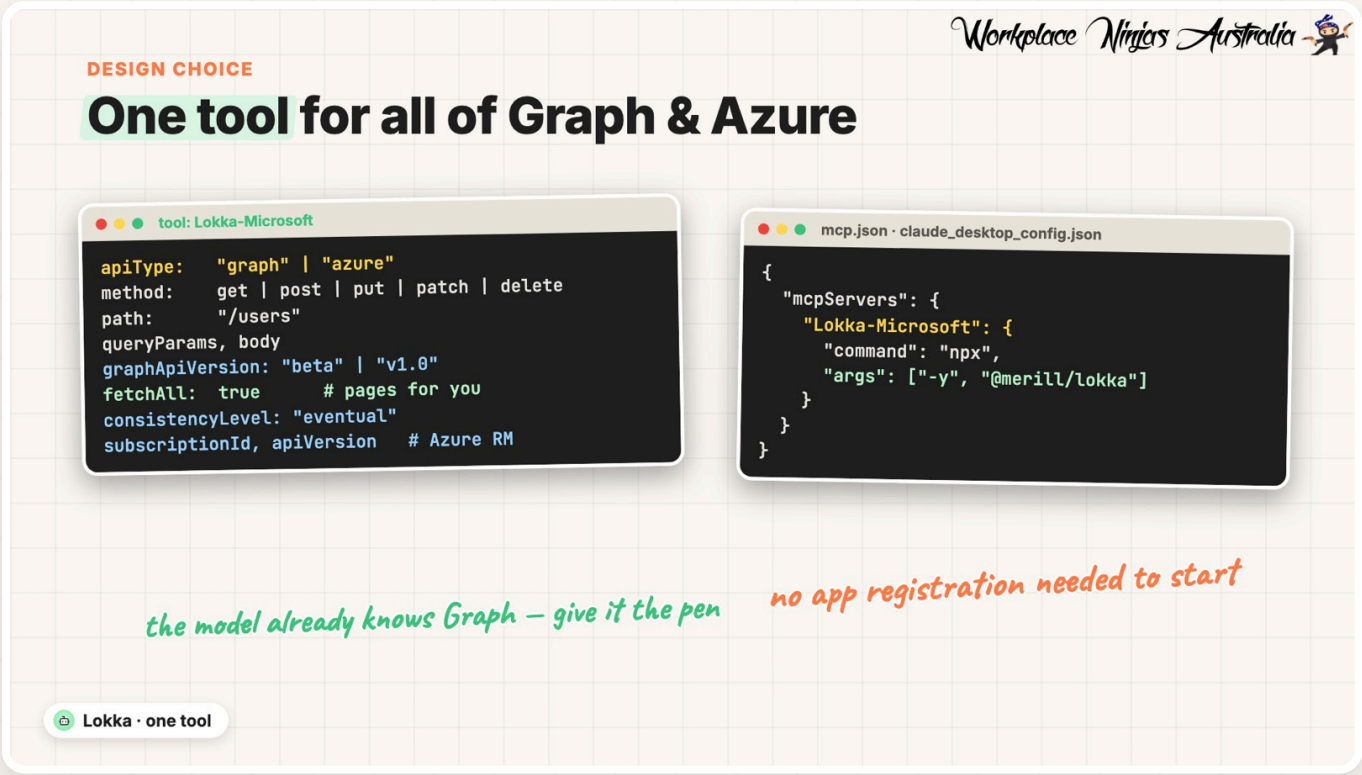
Part 3: Lokka, an MCP server for Microsoft Graph and Azure Resource Manager. A personal open-source project; not a Microsoft product.

LINKS

lokka.dev
`lokka.dev`

Lokka on GitHub
`github.com/merill/lokka`

One tool for all of Graph & Azure



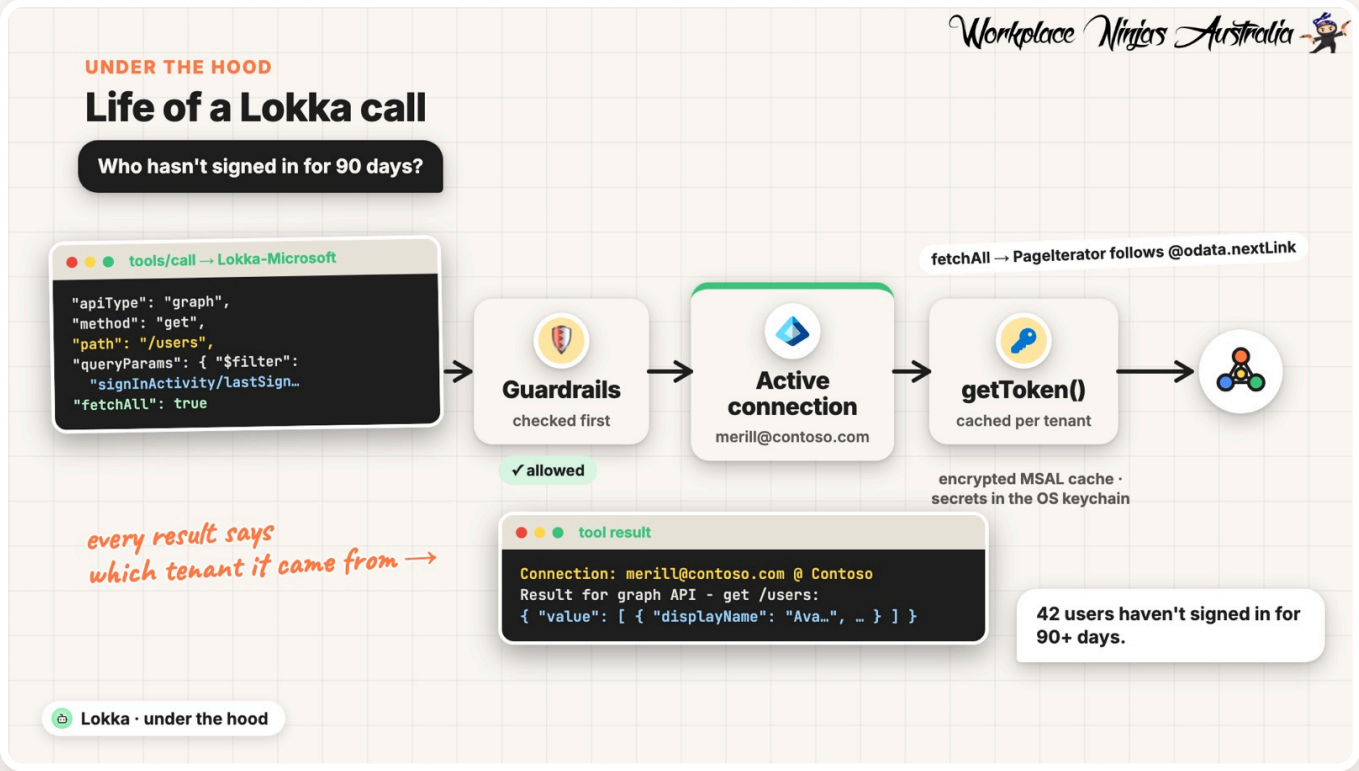
Lokka's main tool, `Lokka-Microsoft`, can call any Graph or Azure Resource Manager endpoint. It takes the API type, method, path, query parameters, body, Graph version and an option to fetch all pages. One generic tool keeps the tool list small and covers new endpoints the day they ship.

To install, add `npx -y @merill/lokka` as an MCP server in your AI app. By default it signs in interactively with the Microsoft Graph PowerShell app, so you don't need your own app registration to start.

LINKS

Install Lokka
`lokka.dev/docs/install`

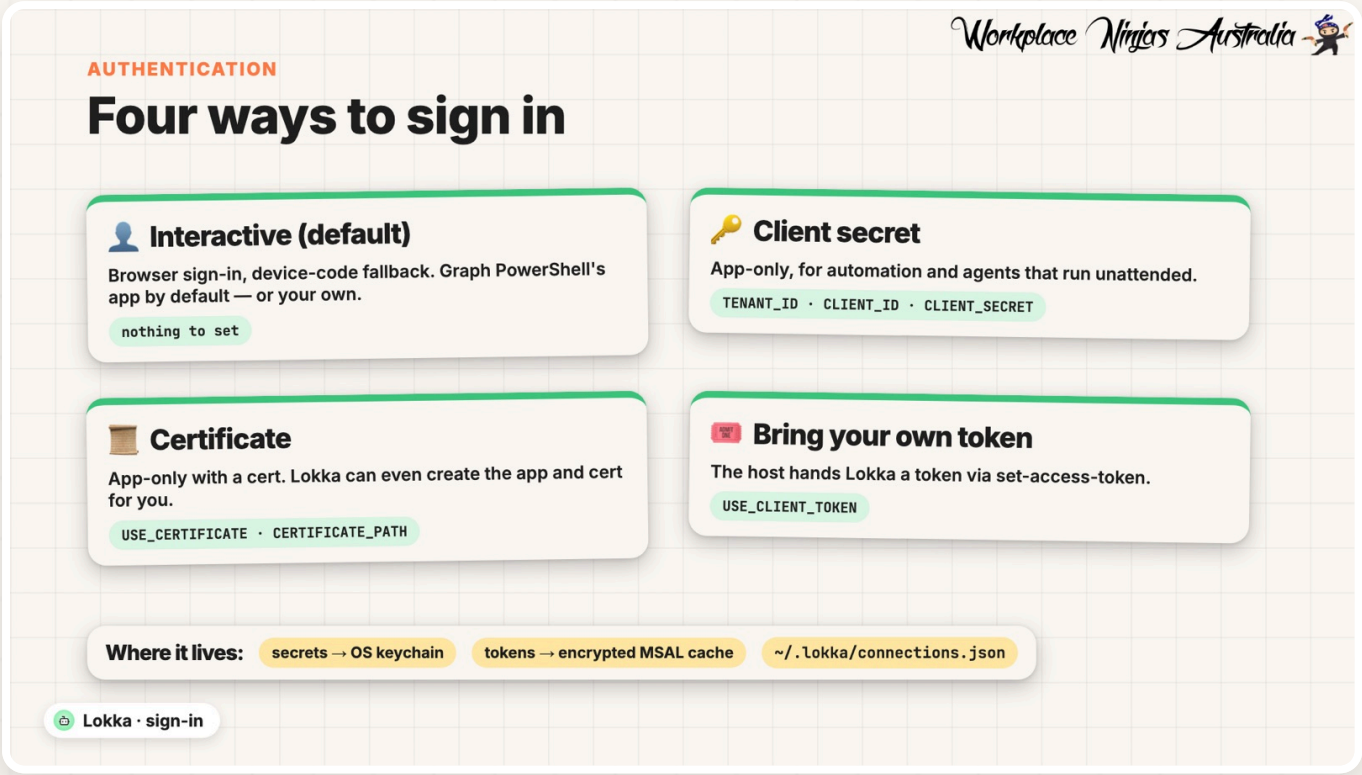
Life of a Lokka call



A question like “Who hasn't signed in for 90 days?” becomes a `tools/call` with a path and filter. Lokka checks its guardrails first, then uses the active connection's Graph client to get a token (cached per tenant), calls Graph, and follows `@odata.nextLink` when you ask for all pages.

Every result is labelled with the connection and tenant it came from, so you and the model always know where the data came from.

Four ways to sign in



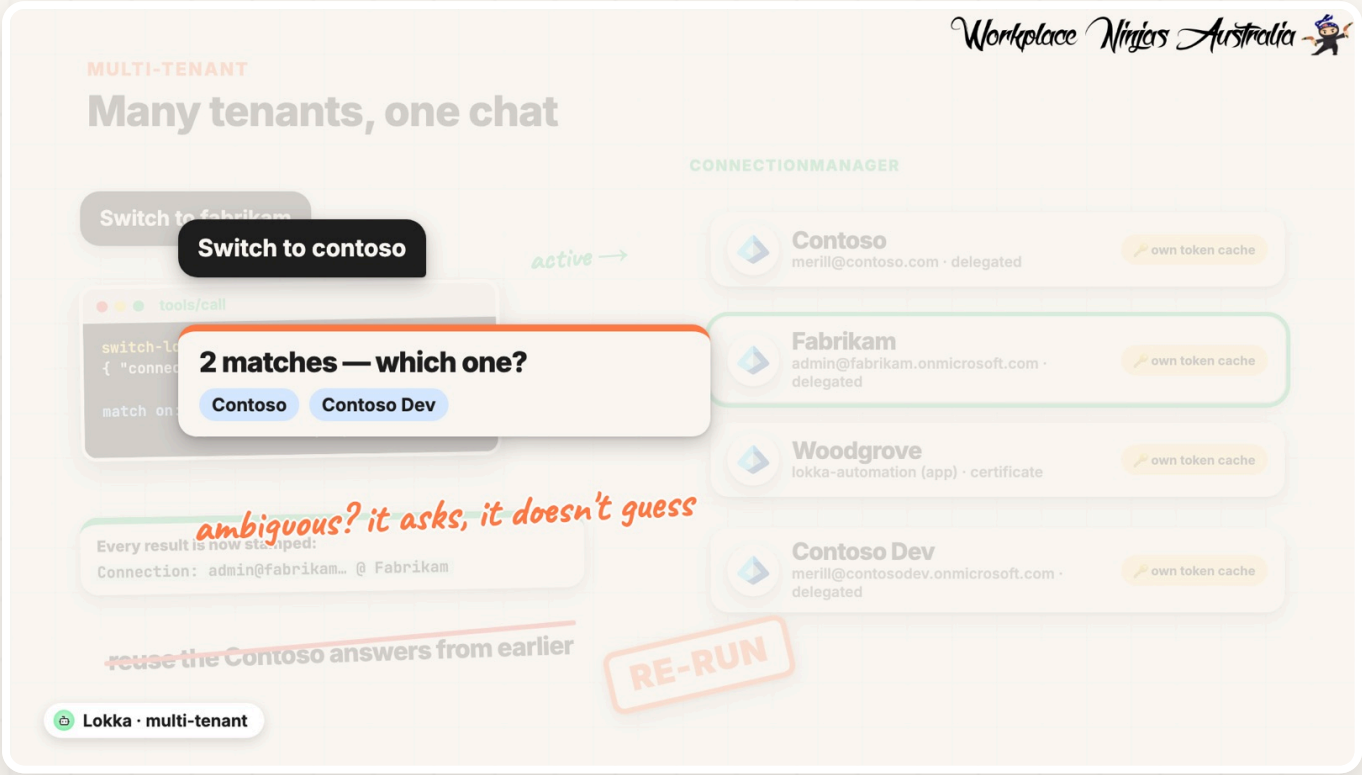
Four sign-in options: **interactive** (the default; browser sign-in with a device-code fallback), **client secret** and **certificate** for app-only automation, and a **token you provide** from the host.

Secrets are stored in your operating system's keychain, tokens in an encrypted token cache, and connection details in `~/.lokka`.

LINKS

Install Lokka
`lokka.dev/docs/install`

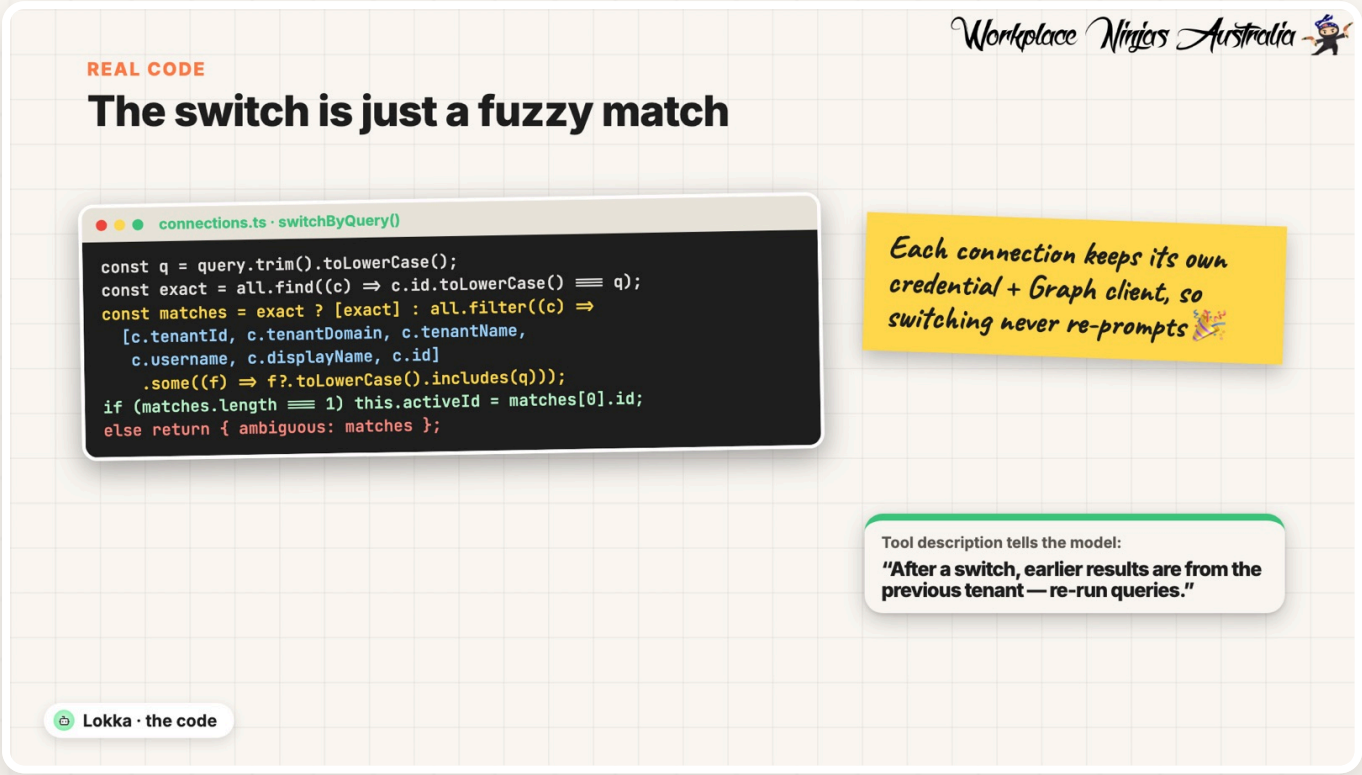
Many tenants, one chat



Lokka can hold several connections at once, across tenants and as users or apps. Each has its own credential and token cache. Say “switch to fabrikam” and Lokka matches the text against tenant ID, domain, tenant name, username, display name and connection ID.

After a switch, results are labelled with the new tenant and the model is told that earlier answers came from the previous tenant. If your text matches more than one connection, Lokka asks which one you meant.

The switch is just a fuzzy match

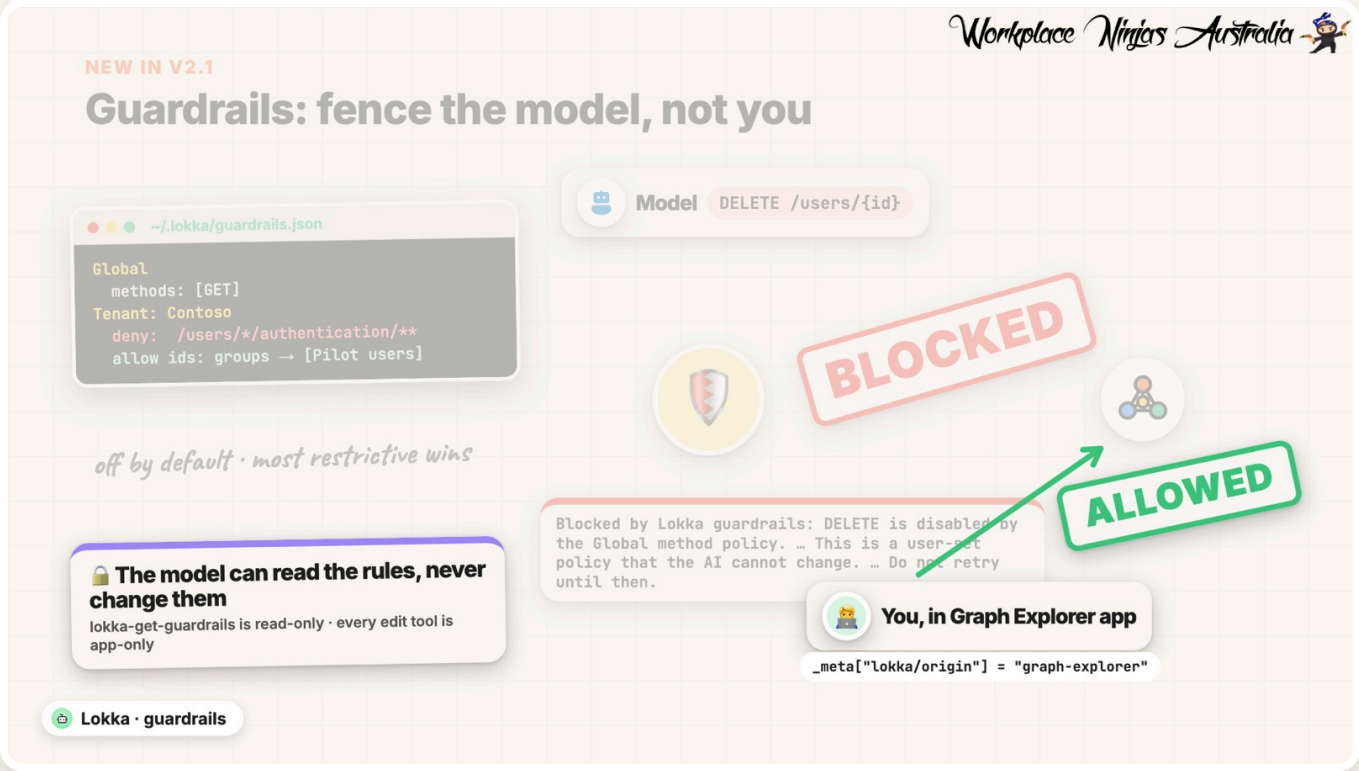


The switch checks for an exact ID first, then a case-insensitive substring match across six fields. One match switches; several return the list of candidates. Because each connection keeps its own client, switching is instant and doesn't prompt you to sign in again.

LINKS

Lokka on GitHub
github.com/merill/lokka

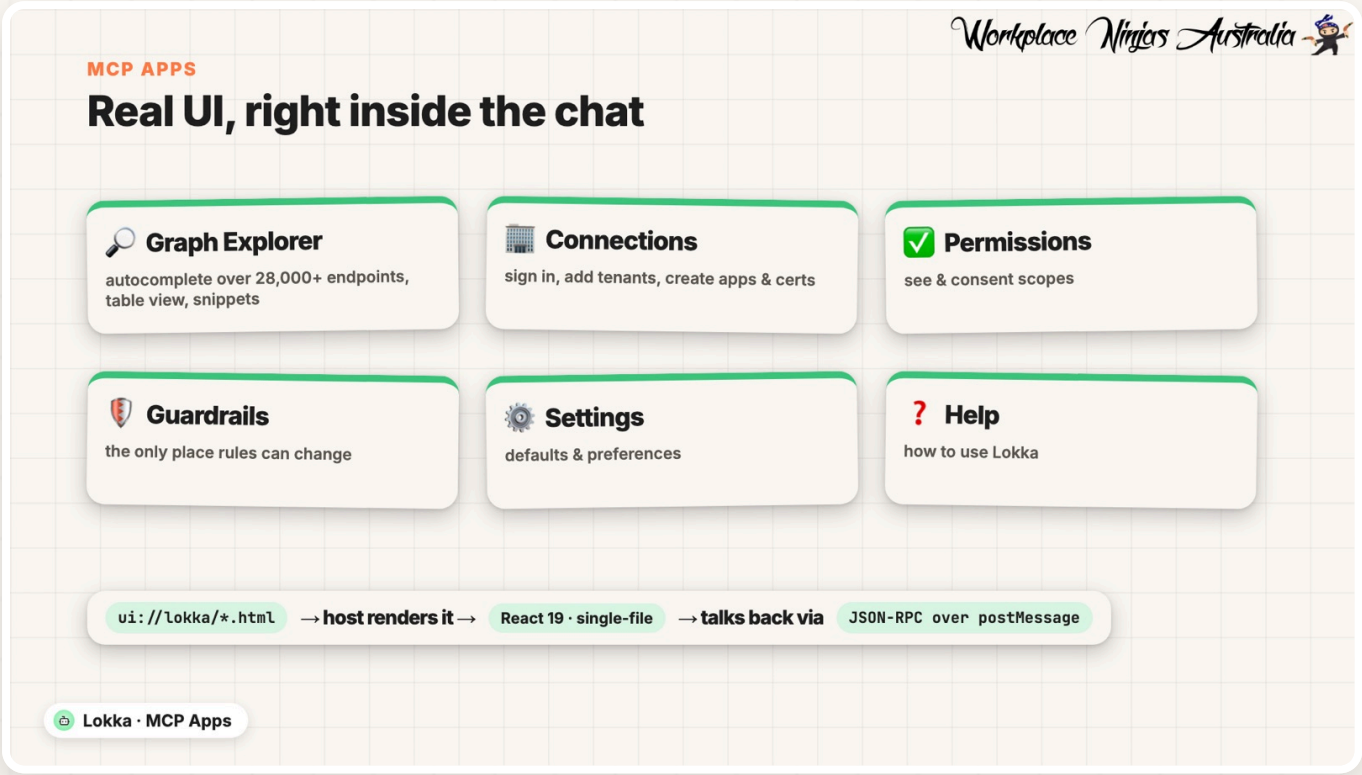
Guardrails



Guardrails (Lokka v2.1) limit what the AI model can do: which HTTP methods it may use, which API paths are allowed or denied, and which specific objects it may touch, globally and per tenant. They're off by default; when on, the default is read-only (GET). The most restrictive rule wins.

Blocked calls fail before anything reaches Microsoft Graph, with a message telling the model not to retry. Calls you make yourself in Lokka's Graph Explorer app aren't blocked, and the model can read the rules but can't change them.

MCP Apps: real UI in the chat

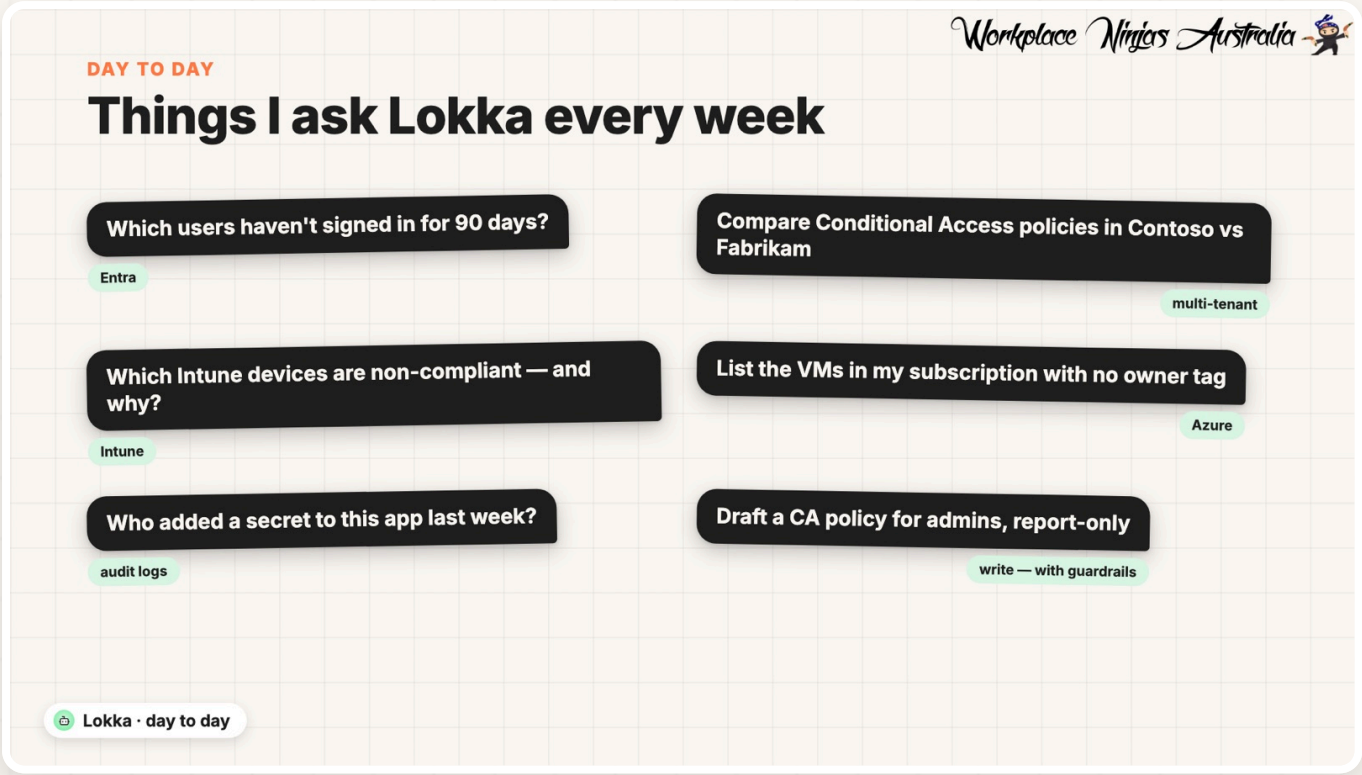


Lokka includes six apps that open inside the chat: Graph Explorer (with autocomplete over 28,000+ endpoints), Connections, Permissions, Guardrails, Settings and Help. They're built with MCP Apps: the host renders them in the conversation and they talk back to Lokka.

LINKS

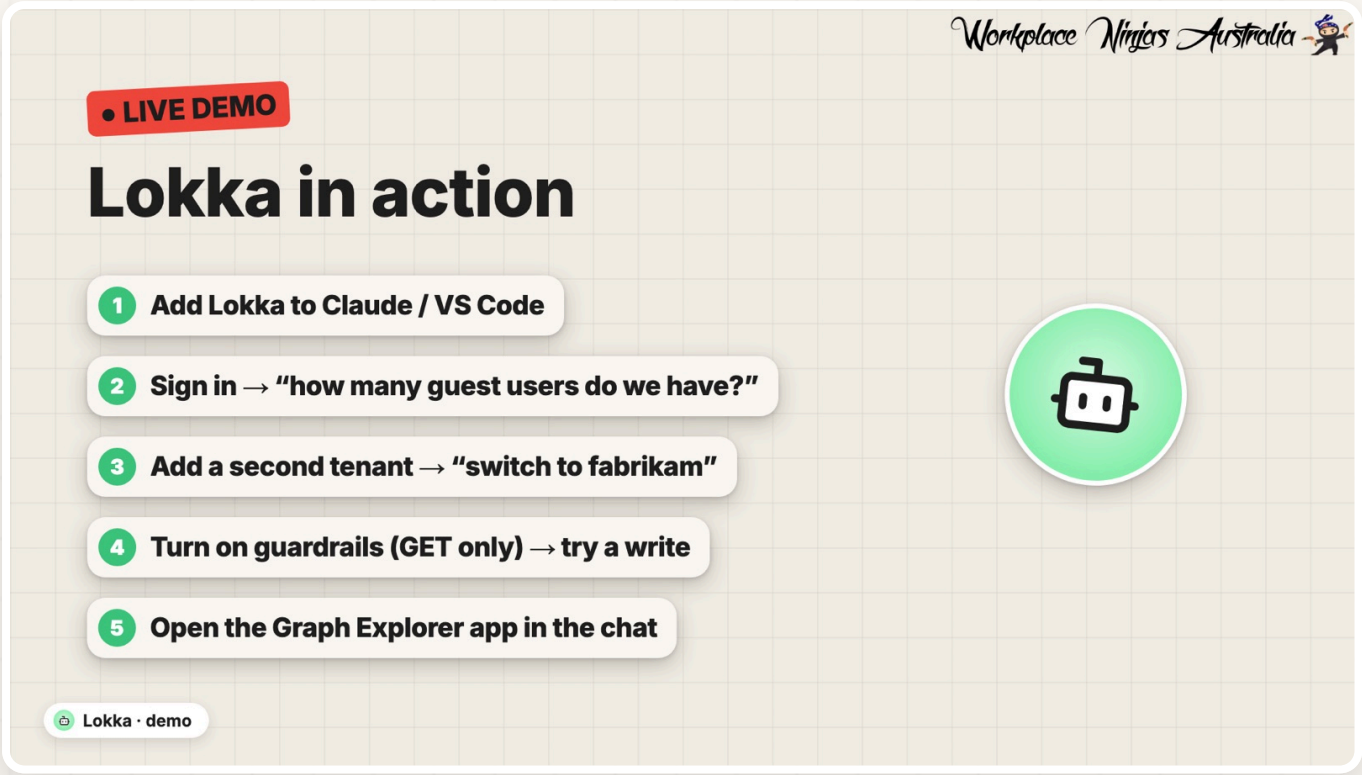
MCP Apps
`modelcontextprotocol.io/docs/extensions/apps`

Things I ask Lokka every week



Prompts to try: “Which users haven't signed in for 90 days?”, “Compare Conditional Access policies in tenant A and tenant B”, “Which Intune devices are non-compliant, and why?”, “List the VMs in my subscription with no owner tag”, “Who added a secret to this app last week?”, “Draft a Conditional Access policy for admins in report-only mode.”

Demo: Lokka



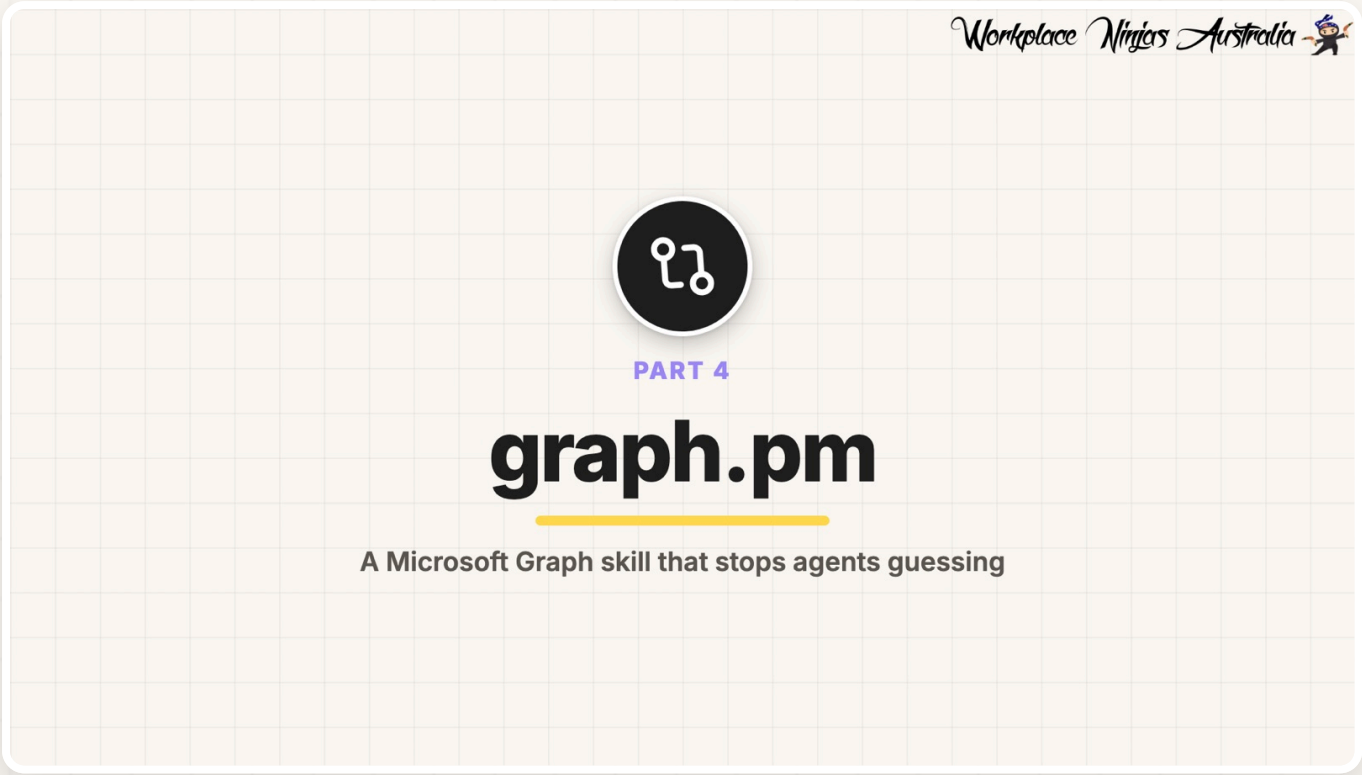
Try it yourself: add Lokka to your AI app and sign in to a test tenant. Ask a question, add a second tenant and switch to it, then turn on read-only guardrails and ask for a change to see it blocked.

Lokka is a proof of concept: use a test tenant and least-privilege permissions.

LINKS

Install Lokka
`lokka.dev/docs/install`

Part 4 — graph.pm

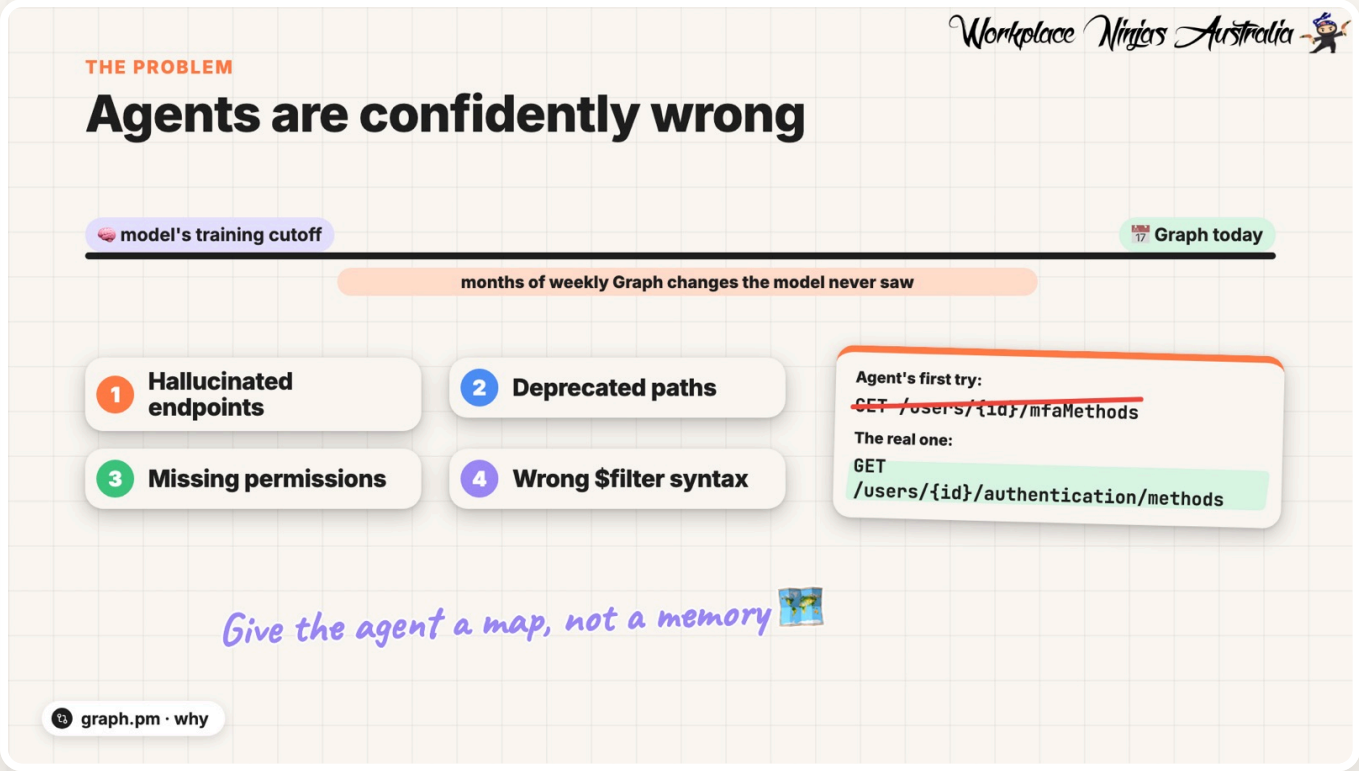


Part 4: graph.pm, a Microsoft Graph skill that gives AI agents an up-to-date map of Graph so they stop guessing.

LINKS

graph.pm
graph.pm

Agents are confidently wrong



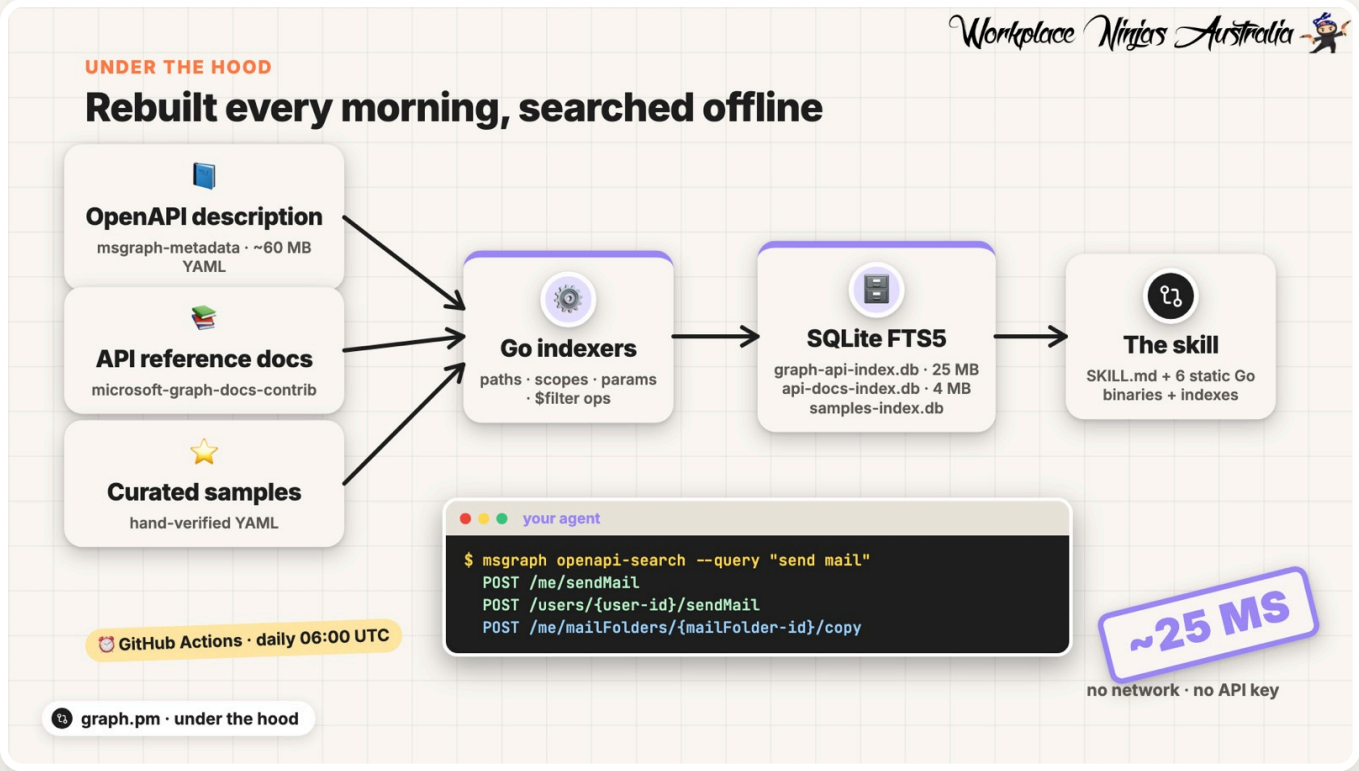
AI models learn from snapshots that are months old, while Microsoft Graph changes every week. So agents invent endpoints, use deprecated paths, miss permissions and get `$filter` syntax wrong.

A classic example: `/users/{id}/mfaMethods` doesn't exist; the real endpoint is `/users/{id}/authentication/methods`.

LINKS

List authentication methods
learn.microsoft.com/en-us/graph/api/authentication-list-methods

Rebuilt every morning, searched offline



Every day a GitHub Action rebuilds graph.pm's indexes from Microsoft's official OpenAPI description, the Graph API reference docs and a set of curated samples. Go indexers extract paths, methods, permissions, query parameters, headers and supported `$filter` operators.

The indexes are SQLite full-text databases that ship inside the skill with small command-line tools for macOS, Linux and Windows. Searches run on your machine in about 25 ms, with no network and no API key.

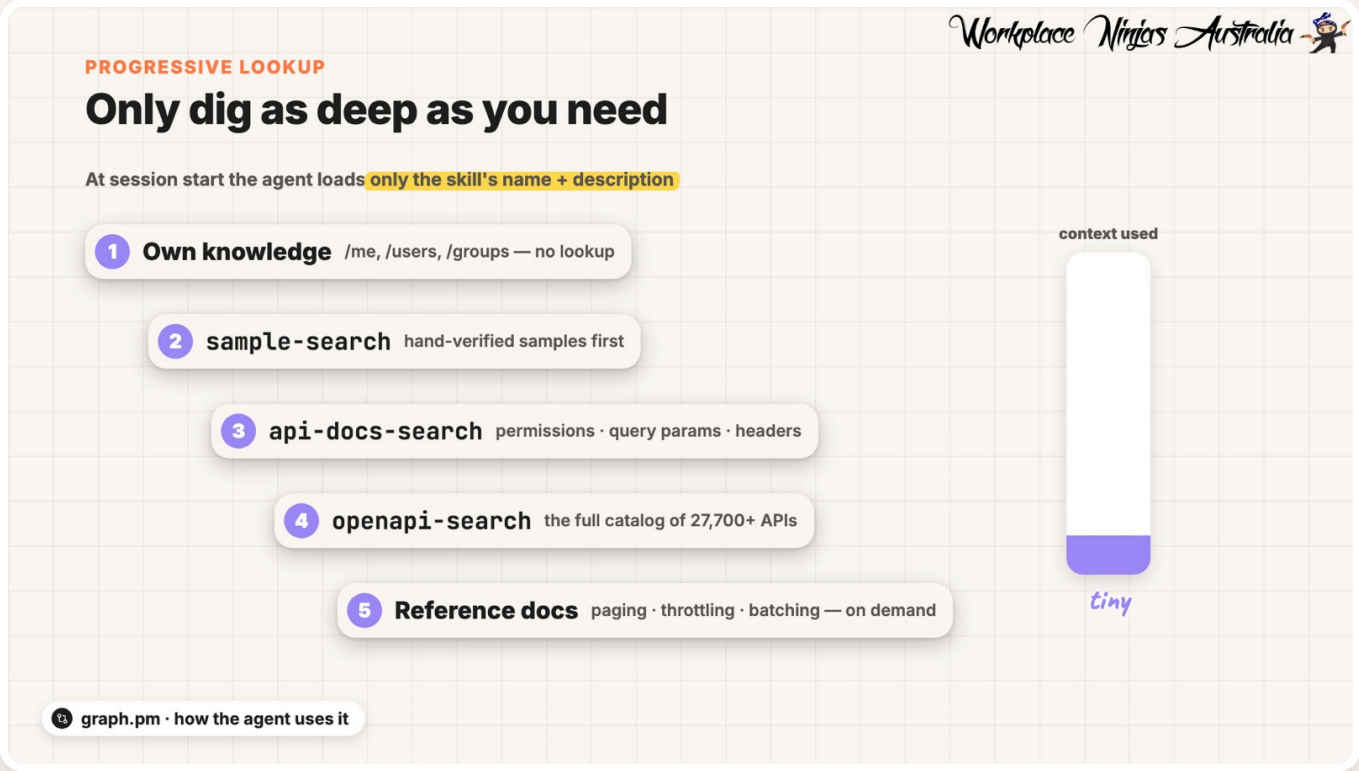
LINKS

msgraph-metadata (OpenAPI)
`github.com/microsoftgraph/msgraph-metadata`

microsoft-graph-docs-contrib
`github.com/microsoftgraph/microsoft-graph-docs-contrib`

graph.pm skill on GitHub
`github.com/merill/msgraph`

Only dig as deep as you need

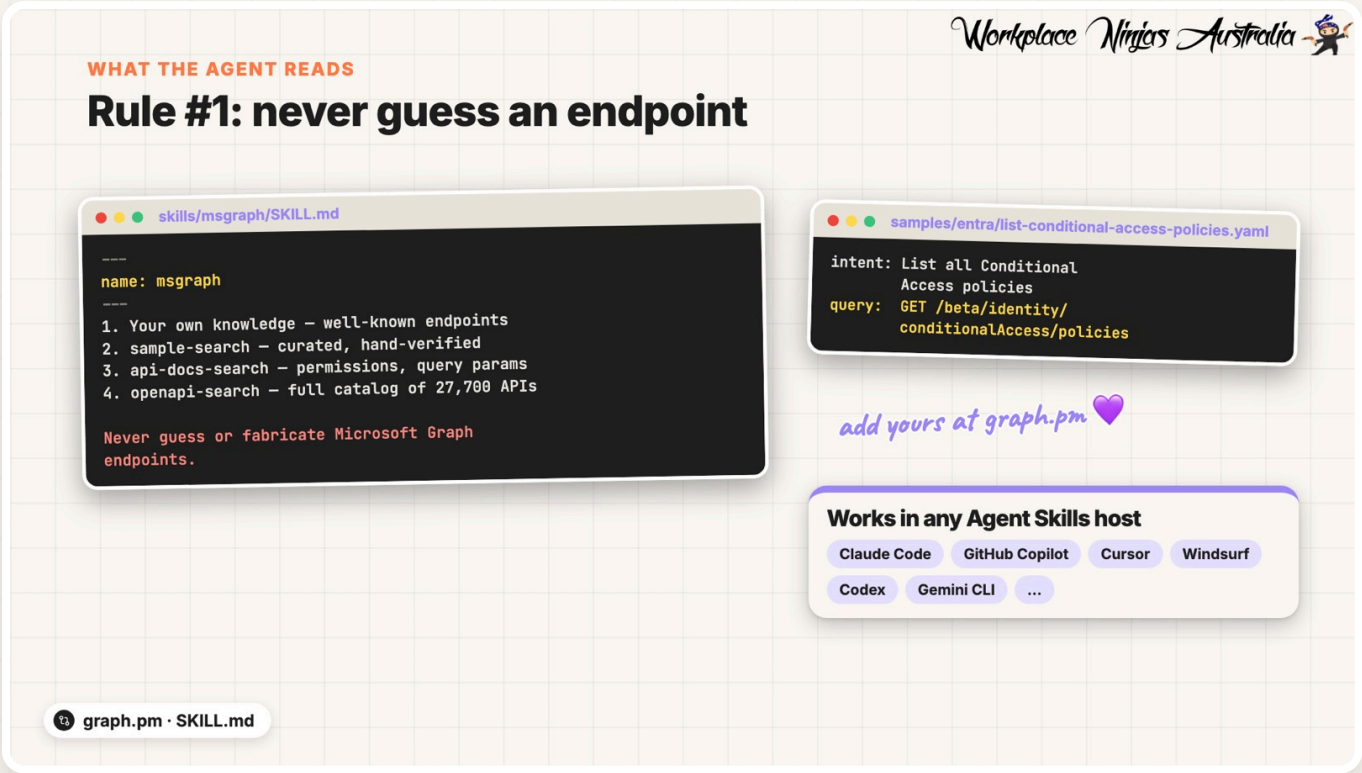


Agent Skills load only their name and description at the start, so they cost almost no context. When the agent needs Graph, the skill tells it to look things up in order: its own knowledge, curated samples, the API docs (permissions, parameters, headers), the full API catalog, then concept docs like paging, throttling and batching, only as needed.

LINKS

Agent Skills
agentskills.io

Rule #1: never guess an endpoint



The skill's instructions (SKILL.md) set the lookup order and one hard rule: never guess or invent a Graph endpoint. Samples are short YAML files with an intent and a query, and anyone can submit one at graph.pm.

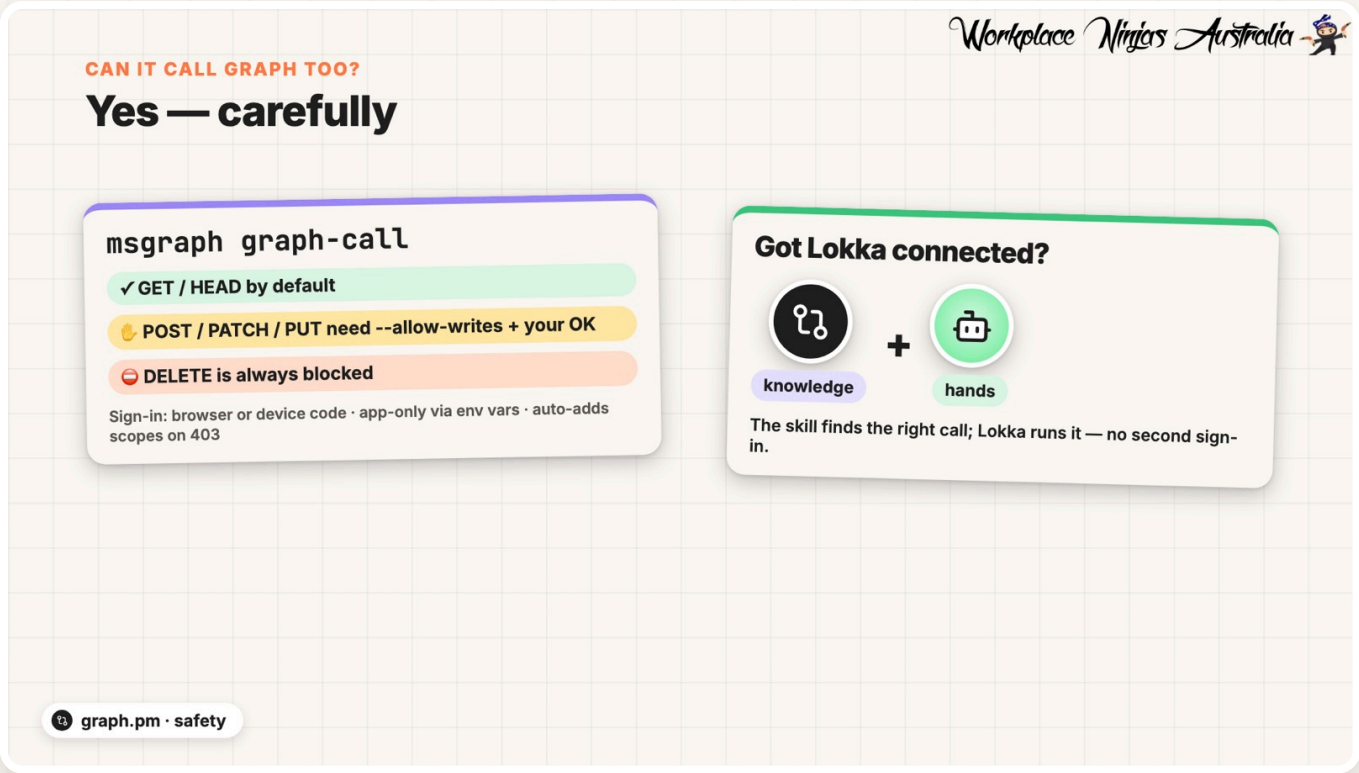
It works in any app that supports Agent Skills, including Claude Code, GitHub Copilot, Cursor, Windsurf, Codex and Gemini CLI.

LINKS

graph.pm
graph.pm

Agent Skills
agentskills.io

Can it call Graph too?

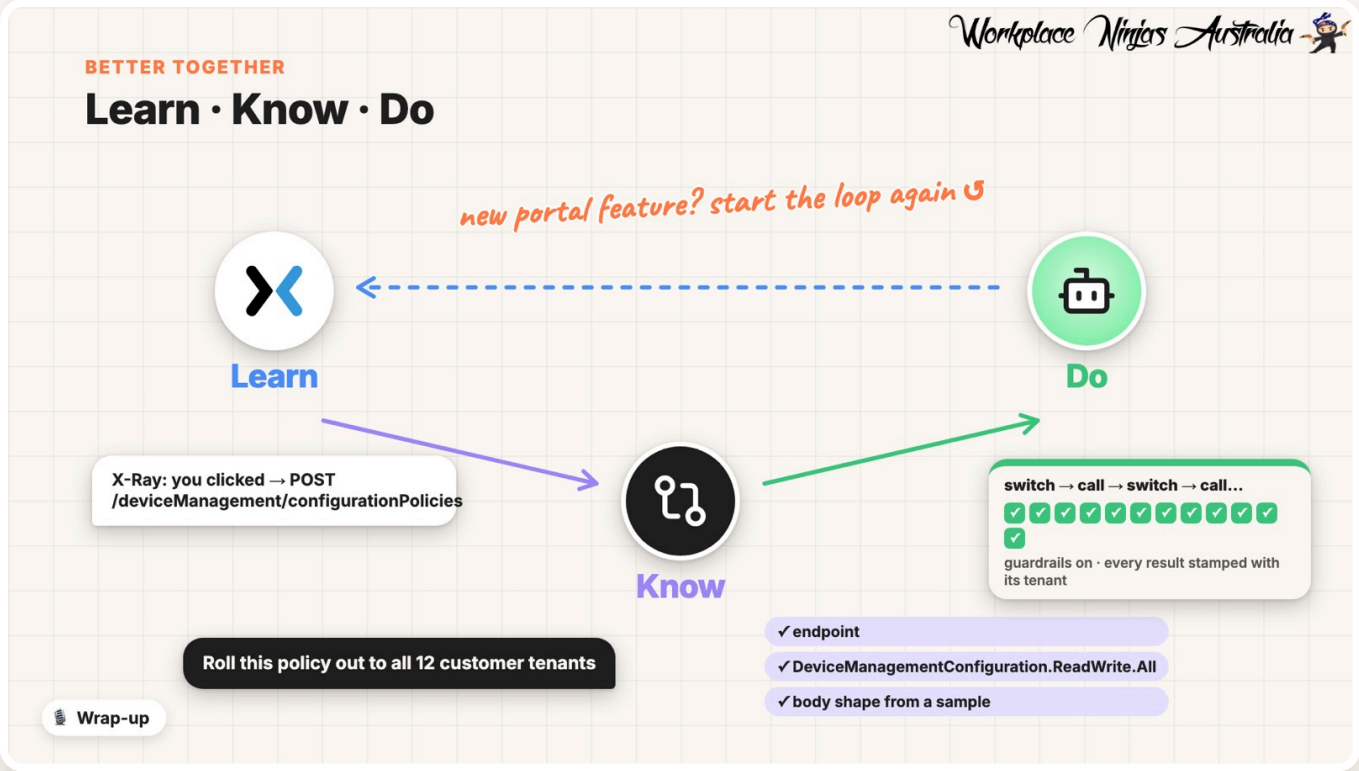


graph.pm can also call Graph itself. It's read-only by default; writes need `--allow-writes` plus your confirmation, and DELETE is always blocked. If you also use Lokka, graph.pm stays the knowledge layer and Lokka makes the calls.

LINKS

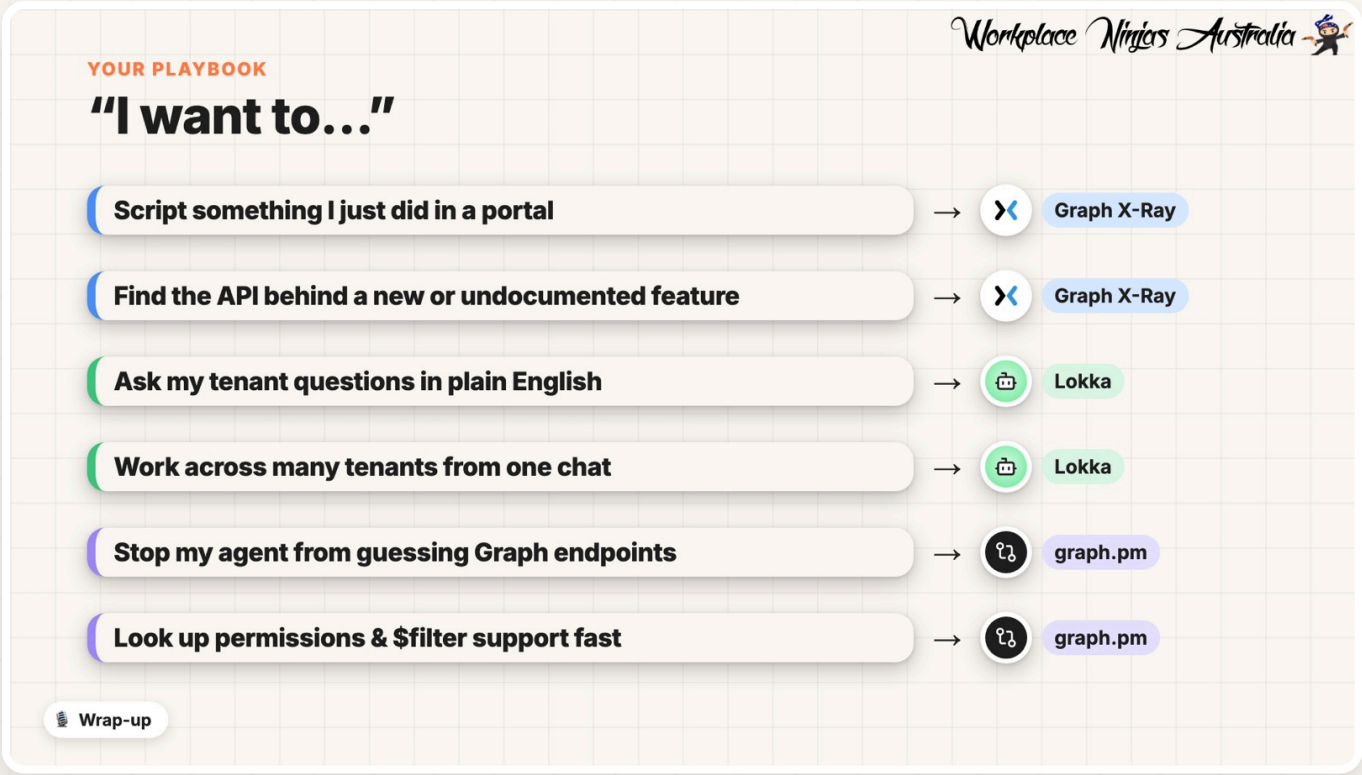
graph.pm skill on GitHub
`github.com/merill/msgraph`

Learn · Know · Do



Learn with Graph X-Ray: do it once in the portal and see the call. **Know** with graph.pm: confirm the right endpoint, permission and request body. **Do** with Lokka: run it across your tenants with guardrails on. When the portal gets a new feature, start again.

Your playbook

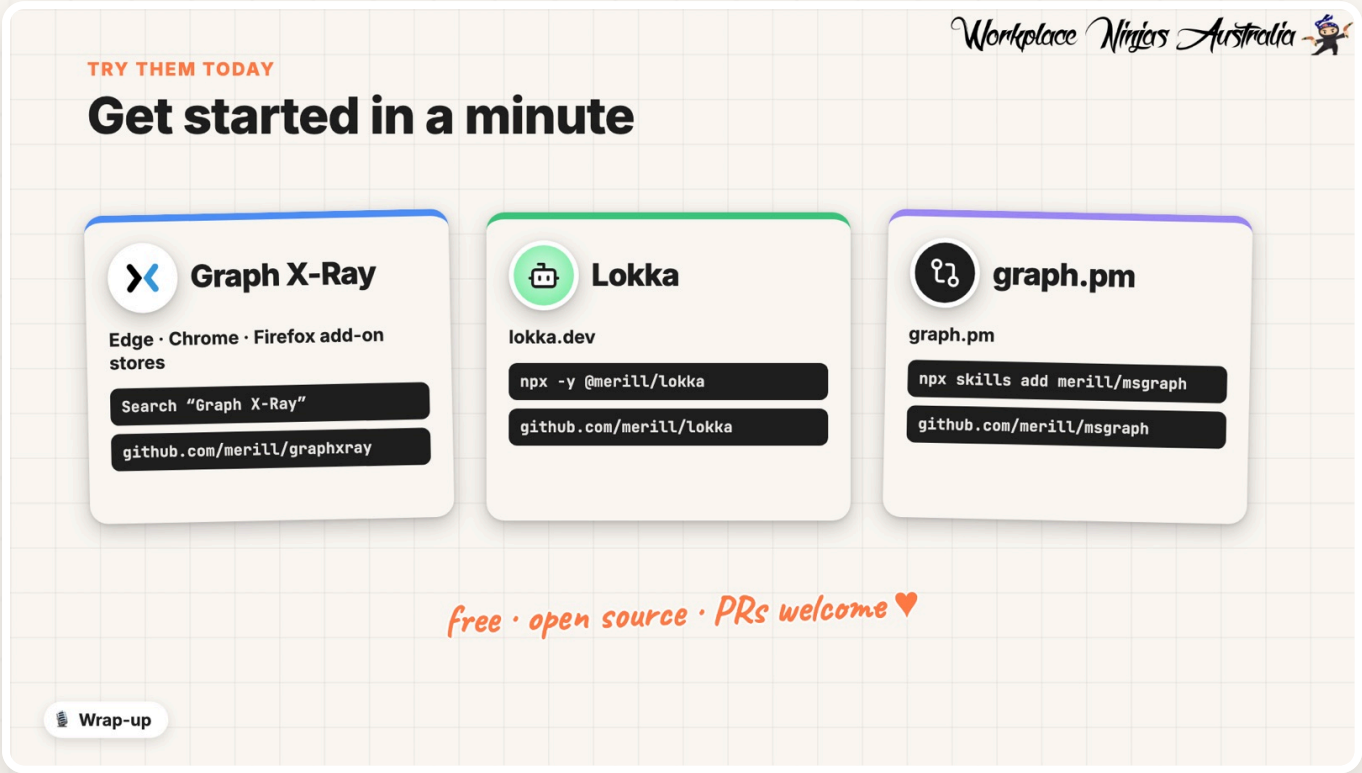


Script something you just did in a portal, or find the API behind a new or undocumented feature: **Graph X-Ray**.

Ask your tenant questions in plain English, or work across many tenants from one chat: **Lokka**.

Stop your agent guessing Graph endpoints, or look up permissions and `$filter` support quickly: **graph.pm**.

Get started in a minute



Graph X-Ray: install from the Edge, Chrome or Firefox add-on store.

Lokka: add `npx -y @merill/lokka` as an MCP server.

graph.pm: run `npx skills add merill/msgraph`.

All three are open source. Issues, samples and pull requests are welcome.

LINKS

Graph X-Ray for Edge
microsoftedge.microsoft.com/addons/detail/graph-xray/oplgganppgjhpihgciiifejplnnpodak

Graph X-Ray for Chrome
chrome.google.com/webstore/detail/graph-x-ray/gdhbldfajbedclijgcmmmobdbnjhnpdh

Graph X-Ray for Firefox
addons.mozilla.org/en-US/firefox/addon/graph-xray

Install Lokka
lokka.dev/docs/install

graph.pm
graph.pm

skills.sh
skills.sh

Questions?



Thank you for coming, and thanks to Workplace Ninjas Australia. Find me at merill.net, or on the Entra.Chat podcast.

LINKS

- merill.net — merill.net
- Entra.Chat — entra.chat